

Artificial Intelligence

Machine Learning & Deep Learning

Artificial Intelligence (AI)

- **Definition:**
The science of making machines “think” and “act” like humans.
- **Goal:**
To build systems that can **perceive, reason, learn, and decide**.
- **Examples:**
 - Google Maps suggesting routes
 - Alexa/Siri voice assistants
 - Self-driving cars

AI is the **umbrella term** under which ML, DL, and GenAI exist.

Machine Learning (ML)

- **Definition:**
A subset of AI where machines **learn from data** instead of being explicitly programmed.
- **Working principle:**
 - Give data → Machine **finds patterns** → Makes predictions.
- **Types of ML:**
 - **Supervised Learning** – Labeled data (e.g., Spam vs Not Spam).
 - **Unsupervised Learning** – No labels, only patterns (e.g., Customer segmentation).

Example: Training a model to predict house prices based on past sales data.

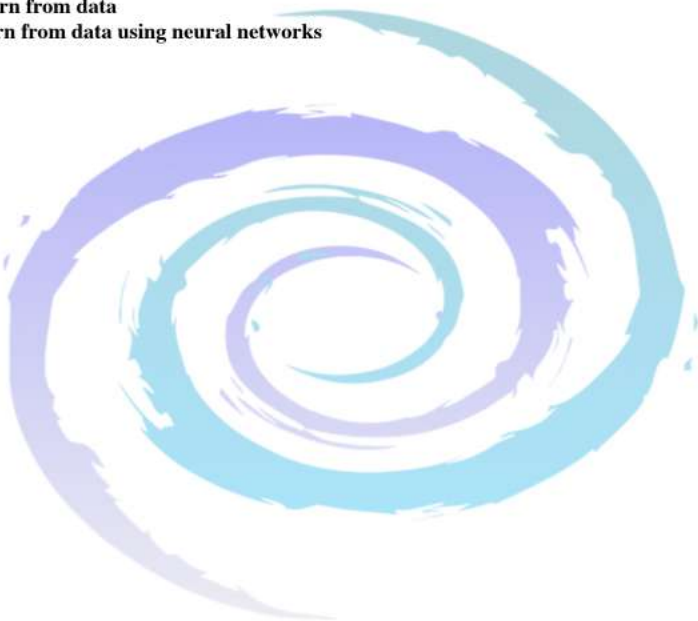
Deep Learning (DL)

- **Definition:**
A special branch of ML using **artificial neural networks (ANNs)** with multiple layers to learn complex patterns.
- **Why “Deep”?**
Because it has **many hidden layers** between input and output.

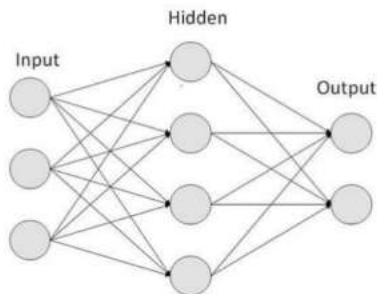
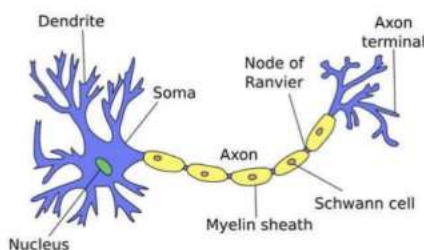
- **Strengths:**
 - Works on images, audio, video, natural language.
 - Can automatically extract features (no manual feature engineering).
- **Examples:**
 - Face recognition on phones
 - Speech-to-text systems
 - Autonomous driving vision systems

ML = Learn from data

DL = Learn from data using neural networks



Biological Neural Network & Artificial Neural Network

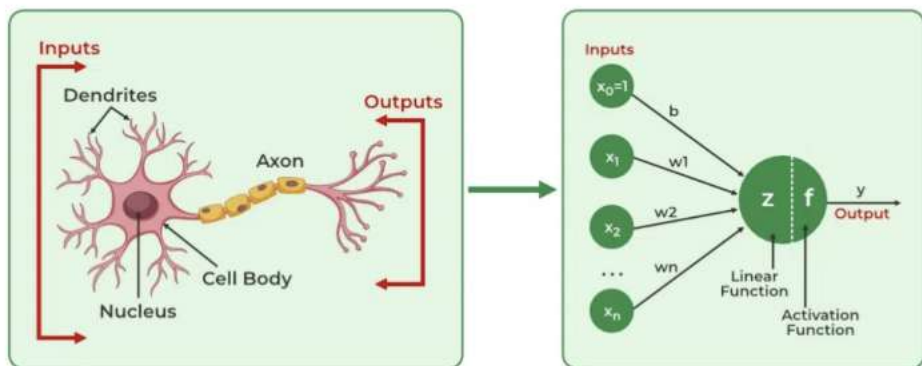


Biological Neural Network (BNN)

- Found in the **human brain and nervous system**.
- Consists of **billions of biological neurons** (brain cells).
- Each neuron is connected to thousands of others via **synapses**.
- Works on **electrochemical signals**.
- **Learning mechanism:** Adjusts the strength of synaptic connections over time (called **synaptic plasticity**).
- Extremely **parallel, adaptive, and energy-efficient**.

Artificial Neural Network (ANN)

- A **mathematical/computational model** inspired by the brain's BNN.
- Consists of **artificial neurons (perceptrons)** arranged in layers.
- Neurons process **numerical data**, not chemical signals.
- **Connections (weights):** Store learned importance of inputs.
- **Learning mechanism:** Adjusts weights using algorithms like **backpropagation** and **gradient descent**.
- Runs on **CPUs/GPUs** with high power consumption.



Biological Neuron (Human Brain)

A **neuron** is the **basic unit of the brain** and nervous system. It processes and transmits information using **electrical and chemical signals**.

Structure

1. **Dendrites** – Branch-like structures that receive signals from other neurons.
2. **Cell Body (Soma)** – Processes the incoming signals.
3. **Axon** – A long extension that transmits signals away to other neurons.
4. **Synapse** – Junction where the axon of one neuron connects to the dendrite of another, using neurotransmitters.

Working

- **Input:** Signals arrive at dendrites.
- **Processing:** Soma integrates signals; if the total signal exceeds a threshold, the neuron “fires.”
- **Output:** Axon carries the electrical impulse to synapses.
- **Learning:** Repeated usage strengthens or weakens synaptic connections (**Synaptic Plasticity**).

Human brain has **~86 billion neurons**, each connecting to thousands of others → massive parallelism.

Artificial Neuron (Perceptron)

An **Artificial Neuron** is a **mathematical model** inspired by biological neurons. It takes inputs, processes them, and produces an output.

Structure

1. **Inputs ($x_1, x_2, \dots, x_n$):** Features/data points fed into the model.
2. **Weights ($w_1, w_2, \dots, w_n$):** Each input has an importance factor.
3. **Bias (b):** A constant added to control the activation threshold.
4. **Summation Function:** Weighted sum of inputs.
$$z = (w_1x_1 + w_2x_2 + \dots + w_nx_n) + b$$
5. **Activation Function:** Decides the output of the neuron (fire or not).

Activation Functions (decide non-linearity)

- **Step function:** If $z > \text{threshold} \rightarrow 1$ else 0
- **Sigmoid:** Smooth curve between 0 and 1
- **Tanh:** Values between -1 and 1
- **ReLU (Rectified Linear Unit):** $\max(0, z)$ (most popular in DL)

Working

1. Inputs are multiplied by weights.
2. Bias is added.
3. Activation function applied.
4. Output sent to next layer.

This mimics “dendrites + soma + axon” of biological neurons.

Biological vs Artificial Neuron

Aspect	Biological Neuron	Artificial Neuron
Unit	Brain cell	Mathematical function
Inputs	Signals from dendrites	Features ($x_1, x_2, \dots$)
Weights	Synaptic strength	Numerical weights ($w_1, w_2, \dots$)
Processing	Soma integrates signals	Weighted sum + bias
Threshold	Fires if strong enough	Activation function
Output	Signal via axon	Predicted value (y)
Learning	Synaptic plasticity	Weight adjustment via algorithms (e.g., backpropagation)

Consider below difference between components of Artificial neuron and Human neuron



Component	Human Neuron	Artificial Neuron
Dendrites	Branch-like extensions that receive signals from other neurons.	Input layer — Receives numerical data from features or previous neurons.
Cell Body (Soma)	Processes incoming signals and determines whether to send a signal forward.	Summation function — Adds up weighted inputs.
Axon	Long projection that transmits electrical impulses to other neurons.	Output value — Sent to next layer's neurons.
Synapse	Connection point where signals are passed chemically to another neuron.	Weights — Determine the strength/importance of the connection.

Activation Function

Activation functions introduce **non-linearity** into neural networks.

Without them, stacking multiple layers would still compute only a **linear transformation**, making deep networks no more powerful than a single linear layer.

- To model **non-linear** relationships in data (e.g., XOR, images, language).
- To control **neuron firing range** (probabilities, bounded outputs).
- To help **optimization** by shaping gradients during backpropagation.

Sigmoid (Logistic) Activation

Formula:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- **Range:** (0, 1) — interpretable as probability.
- **Pros:** Smooth, interpretable as probability, useful for binary classification output.
- **Cons:** Vanishing gradient problem, not zero-centered.
- **Best use:** Output layer of **binary classification problems**.

ReLU (Rectified Linear Unit)

Formula:

$$ReLU(z) = \max(0, z)$$

- **Range:** [0, ∞)
- **Pros:** Solves vanishing gradient (for positive z), simple & fast, sparse activation.
- **Cons:** “Dying ReLU” problem (neuron stuck at 0 for all inputs).
- **Best use:** Default for **hidden layers** in deep networks.

Activation Functions in ANN Flow

$$z = Wx + b, \quad a = f(z)$$

$f(z)$ is the activation function (Sigmoid, ReLU, etc.).

Without $f(z)$, networks remain linear $\rightarrow$ limited power.

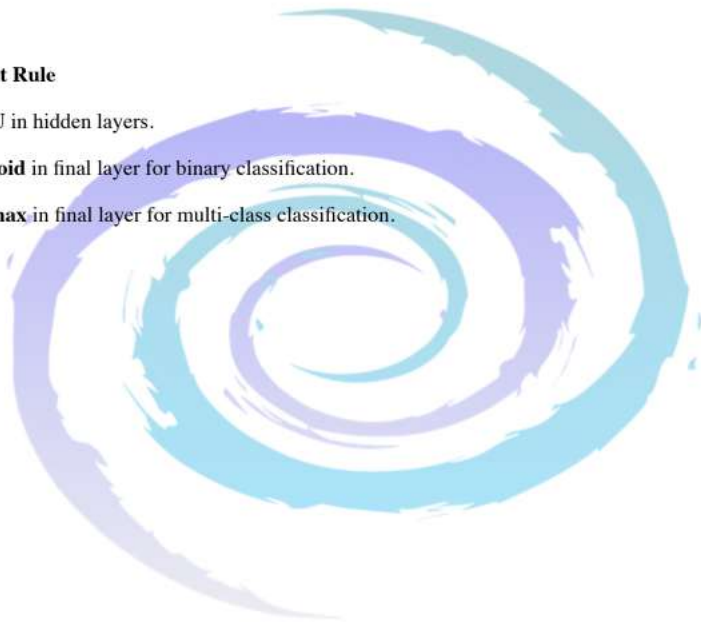
During backpropagation, gradients flow through $f'(z)$. If derivative ≈ 0 (Sigmoid saturation), learning slows.

Important Rule

Use **ReLU** in hidden layers.

Use **Sigmoid** in final layer for binary classification.

Use **Softmax** in final layer for multi-class classification.



Sigmoid function as a activation function

Page 3

```
bias = 0.5          # Bias term

# Run the neuron forward pass
z, y_hat = Marvellous_neuron_forward(inputs, weights, bias)

# Plot sigmoid curve
plot_sigmoid()

if __name__ == "__main__":
    main()
```



Consider below application to demonstrate the concept of ReLU function as a activation function

```
import matplotlib.pyplot as plt
import numpy as np

# ReLU activation function
def relu(z):
    return max(0, z)

# ---- Neuron calculation ----
def Marvellous_neuron_forward(inputs, weights, bias):
    # 1) Display inputs and weights
    print("Inputs (x): ", inputs)
    print("Weights (w):", weights)
    print("Bias (b): ", bias)

    # 2) Summation  $z = w \cdot x + b$ 
    z = sum(w * x for w, x in zip(weights, inputs)) + bias
    print("Summation ( $z = w \cdot x + b$ ):", z)

    # 3) Activation function output
    y_hat = relu(z)
    print("Activation Function: ReLU")
    print("Output ( $\hat{y} = \text{relu}(z)$ ):", y_hat)

    return z, y_hat

def plot_relu():
    z_values = np.linspace(-10, 10, 200) # range of z values
    relu_values = np.maximum(0, z_values)

    plt.figure(figsize=(8, 5))
    plt.plot(z_values, relu_values, label="ReLU", linewidth=2, color="green")
    plt.axhline(y=0, color="black", linewidth=0.5)
    plt.axvline(x=0, color="gray", linestyle="--")
    plt.title("ReLU Activation Function", fontsize=16)
    plt.xlabel("Summation (z)", fontsize=14)
    plt.ylabel("Activation Output", fontsize=14)
    plt.grid(True, linestyle="--", alpha=0.6)
    plt.legend()
    plt.show()

def main():
    # Example inputs, weights, and bias
    inputs = [1.0, 2.0, 3.0] # Example input features
    weights = [0.6, 0.4, -0.2] # Weights for each input
    bias = 0.5 # Bias term

    # Run the neuron forward pass
    z, y_hat = Marvellous_neuron_forward(inputs, weights, bias)

    plot_relu()

if __name__ == "__main__":
    main()
```

Consider below application to demonstrate the comparison in Sigmoid and ReLU function as a activation function

```
import math
import matplotlib.pyplot as plt
import numpy as np

# -----
# Activation functions
# -----
def sigmoid(z):
    """Sigmoid function: squashes values to (0,1)."""
    return 1 / (1 + math.exp(-z))

def relu(z):
    """ReLU function: outputs z if positive, else 0."""
    return max(0, z)

# -----
# Neuron calculation
# -----
def Marvellous_neuron_forward(inputs, weights, bias, activation_func):
    # 1) Display inputs and weights
    print("Inputs (x): ", inputs)
    print("Weights (w):", weights)
    print("Bias (b): ", bias)

    # 2) Summation
    z = sum(w * x for w, x in zip(weights, inputs)) + bias
    print("Summation (z = w·x + b):", z)

    # 3) Activation
    y_hat = activation_func(z)
    print(f"Activation Function: {activation_func.__name__}")
    print(f"Output (ŷ): {y_hat}\n")

    return z, y_hat

# -----
# Plot Sigmoid and ReLU
# -----
def plot_sigmoid_relu():
    z_values = np.linspace(-10, 10, 200)
    sigmoid_values = 1 / (1 + np.exp(-z_values))
    relu_values = np.maximum(0, z_values)

    plt.figure(figsize=(8, 5))
    plt.plot(z_values, sigmoid_values, label="Sigmoid", linewidth=2, color="blue")
    plt.plot(z_values, relu_values, label="ReLU", linewidth=2, color="green")
    plt.axhline(y=0, color="black", linewidth=0.5)
    plt.axhline(y=1, color="black", linewidth=0.5)
    plt.axvline(x=0, color="gray", linestyle="--")
    plt.title("Sigmoid vs ReLU Activation Functions", fontsize=16)
    plt.xlabel("Summation (z)", fontsize=14)
    plt.ylabel("Activation Output", fontsize=14)
    plt.grid(True, linestyle="--", alpha=0.6)
```

```
plt.legend()
plt.show()

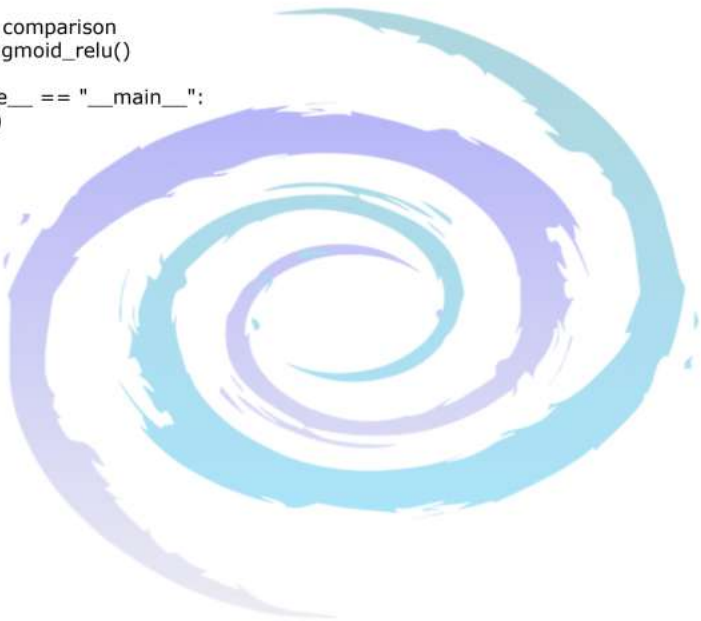
def main():
    # Example inputs, weights, bias
    inputs = [1.0, 2.0, 3.0]
    weights = [0.6, 0.4, -0.2]
    bias = 0.5

    print("=== Sigmoid Neuron ===")
    Marvellous_neuron_forward(inputs, weights, bias, sigmoid)

    print("=== ReLU Neuron ===")
    Marvellous_neuron_forward(inputs, weights, bias, relu)

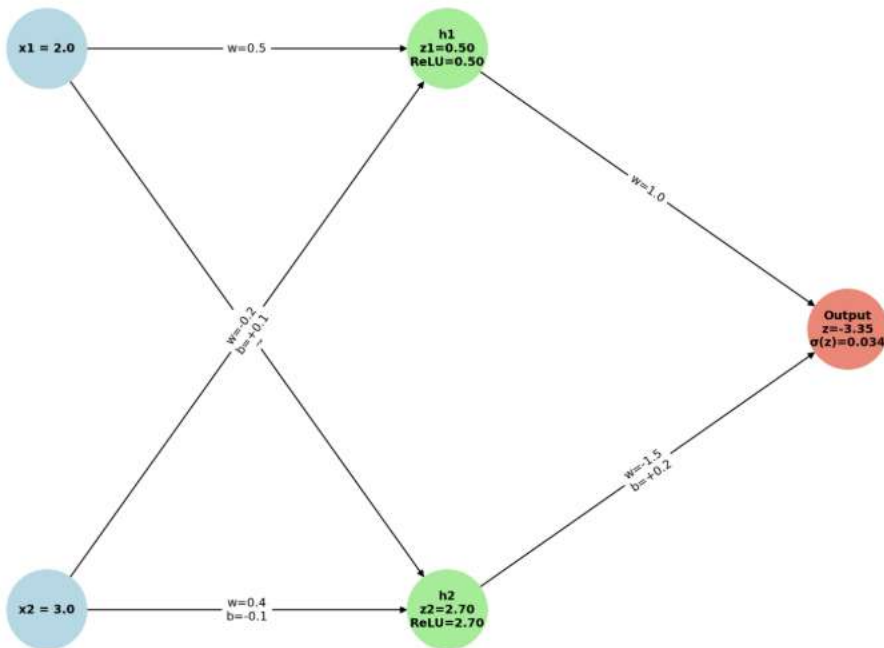
    # Plot comparison
    plot_sigmoid_relu()

if __name__ == "__main__":
    main()
```



Application which implements 2 Layered Neural Network Design

2-Layer Neural Network with Values (Inputs → Hidden ReLU → Output Sigmoid)



```
import math
```

```
# ----- Activation Functions -----
```

```
def relu(x):
    return max(0, x)
```

```
def sigmoid(z):
    return 1 / (1 + math.exp(-z))
```

```
# ----- Forward Pass with Step-by-Step Printing -----
```

```
def Marvellous_forward_pass(inputs):
    print("=== INPUT LAYER ===")
    print(f"Inputs: x1 = {inputs[0]}, x2 = {inputs[1]}")
```

```
# ----- Hidden Layer -----
```

```
weights_hidden = [
    [0.5, -0.2], # Neuron 1 weights
    [0.8, 0.4]  # Neuron 2 weights
]
```

```
bias_hidden = [0.1, -0.1] # Bias for each hidden neuron
```

```

hidden_outputs = []
print("\n=== HIDDEN LAYER ===")
for i in range(len(weights_hidden)):
    print(f"\nNeuron {i+1}:")
    w = weights_hidden[i]
    b = bias_hidden[i]

    # Detailed multiplication steps
    print(f" Step 1: Multiply inputs by weights:")
    print(f"   ({w[0]} * {inputs[0]}) = {w[0] * inputs[0]:.3f}")
    print(f"   ({w[1]} * {inputs[1]}) = {w[1] * inputs[1]:.3f}")

    # Summation
    z = sum(w_j * x_j for w_j, x_j in zip(w, inputs)) + b
    print(f" Step 2: Add results and bias {b}: z = {z:.3f}")

    # Activation
    a = relu(z)
    print(f" Step 3: Apply ReLU: max(0, {z:.3f}) = {a:.3f}")

    hidden_outputs.append(a)

# ----- Output Layer -----
weights_output = [1.0, -1.5] # Weights from hidden to output
bias_output = 0.2

print("\n=== OUTPUT LAYER ===")
print(" Step 1: Multiply hidden outputs by weights:")
print(f"   ({weights_output[0]} * {hidden_outputs[0]}) = {weights_output[0] * hidden_outputs[0]:.3f}")
print(f"   ({weights_output[1]} * {hidden_outputs[1]}) = {weights_output[1] * hidden_outputs[1]:.3f}")

z_out = sum(w_o * h for w_o, h in zip(weights_output, hidden_outputs)) + bias_output
print(f" Step 2: Add results and bias {bias_output}: z = {z_out:.3f}")

y_hat = sigmoid(z_out)
print(f" Step 3: Apply Sigmoid: 1 / (1 + e^(-{z_out:.3f})) = {y_hat:.3f}")

print(f"\nFinal Output: {y_hat:.3f} → {y_hat*100:.2f}% confidence in Positive Class")

if __name__ == "__main__":
    inputs = [2.0, 3.0] # Example input features
    Marvellous_forward_pass(inputs)

```



```

marvellous@Marvellouss-MacBook-Pro Codes % python3 MultilayerANN.py
=== INPUT LAYER ===
Inputs: x1 = 2.0, x2 = 3.0

=== HIDDEN LAYER ===

Neuron 1:
Step 1: Multiply inputs by weights:
(0.5 * 2.0) = 1.000
(-0.2 * 3.0) = -0.600
Step 2: Add results and bias 0.1: z = 0.500
Step 3: Apply ReLU: max(0, 0.500) = 0.500

Neuron 2:
Step 1: Multiply inputs by weights:
(0.8 * 2.0) = 1.600
(0.4 * 3.0) = 1.200
Step 2: Add results and bias -0.1: z = 2.700
Step 3: Apply ReLU: max(0, 2.700) = 2.700

=== OUTPUT LAYER ===
Step 1: Multiply hidden outputs by weights:
(1.0 * 0.4999999999999999) = 0.500
(-1.5 * 2.7) = -4.050
Step 2: Add results and bias 0.2: z = -3.350
Step 3: Apply Sigmoid: 1 / (1 + e^(-3.350)) = 0.034

Final Output: 0.034 → 3.39% confidence in Positive Class
marvellous@Marvellouss-MacBook-Pro Codes %

```

Explanation of above code

It performs a **single forward pass** through a tiny neural network with:

- **Inputs:** x1, x2
- **Hidden layer:** 2 neurons with **ReLU**
- **Output layer:** 1 neuron with **Sigmoid** and prints every intermediate computation.

Activations

```

def relu(x):      return max(0, x)
def sigmoid(z):   return 1 / (1 + math.exp(-z))

```

- **ReLU** keeps positive values and zeroes out negatives → helps non-linearity and avoids vanishing gradients for $z > 0$
- **Sigmoid** maps any real number to (0,1) → interpretable as a probability for the “positive” class.

The forward function

```

def Marvellous_forward_pass(inputs):
    print("=== INPUT LAYER ===")

```



```
print(f"Inputs: x1 = {inputs[0]}, x2 = {inputs[1]}")
```

- inputs is a 2-item list [x1, x2]. In `__main__` you pass [2.0, 3.0].

Hidden layer setup

```
weights_hidden = [
    [0.5, -0.2], # Neuron 1 weights (w11,w12)
    [0.8, 0.4]  # Neuron 2 weights (w21,w22)
]
```

```
bias_hidden = [0.1, -0.1] # b1, b2
```

- Two neurons. Each has **2 weights** (one per input) and **one bias**.
- We'll compute each hidden neuron's pre-activation z and activation a

Hidden layer — per-neuron math (matches printed steps)

For Neuron 1

```
w = [0.5, -0.2]; b = 0.1
z1 = (0.5*2.0) + (-0.2*3.0) + 0.1
    = 1.0      + (-0.6)      + 0.1
    = 0.5
a1 = ReLU(z1) = max(0, 0.5) = 0.5
```

For Neuron 2

```
w = [0.8, 0.4]; b = -0.1
z2 = (0.8*2.0) + (0.4*3.0) + (-0.1)
    = 1.6      + 1.2      - 0.1
    = 2.7
a2 = ReLU(z2) = max(0, 2.7) = 2.7
```

The code stores these in `hidden_outputs = [a1, a2] = [0.5, 2.7]`.

Output layer — weighted sum then Sigmoid

```
weights_output = [1.0, -1.5] # v1 (from h1), v2 (from h2)
bias_output = 0.2
```

Compute pre-activation z and final output $y^{\wedge}$:

```
z_out = (1.0*0.5) + (-1.5*2.7) + 0.2
       = 0.5      + (-4.05)      + 0.2
       = -3.35
```

```
y_hat = sigmoid(z_out) = 1 / (1 + e^{3.35}) ≈ 0.034
```

So the network outputs ~ 0.034 , i.e., **3.4% confidence** in the positive class for inputs

[2.0 , 3.0]

- **ReLU in the hidden layer:** keeps the layer piece-wise linear but avoids saturating small gradients for positive inputs; encourages sparse activations.
- **Sigmoid at output:** converts the final score into a probability for a **binary** decision.

Final conclusions :

Interpretation: low probability $\rightarrow$ model leans toward the **negative** class.



3. Broadcasting:

- Bias vectors applied across a batch of inputs.
- Example: Adding a 1D tensor (bias) to a 2D tensor (matrix of activations).

4. Automatic Differentiation:

- Gradients wrt tensor values are essential for training.

5. Real Applications:

- **Computer Vision:** Images as tensors (Convolutional Neural Networks).
- **NLP:** Sentences converted to tensor embeddings (Transformers).
- **Speech Recognition:** Audio signals represented as tensors.
- **Reinforcement Learning:** States, actions, and rewards stored as tensors.

◆ Example Flow in Deep Learning with Tensors

👉 Suppose we build a simple image classifier:

- Input: Batch of 32 images, each 28×28 grayscale → Tensor shape [32 , 28 , 28 , 1].
- Layer 1: Convolution filter (3×3×1×32) → Extracts features.
- Layer 2: Dense Layer (weights [6272, 128]) → Flattened features to hidden neurons.
- Output: Dense Layer (weights [128, 10]) → Probability tensor for 10 classes (digits 0–9).

At each step:

- **Forward pass:** Tensors flow through layers.
- **Backward pass:** Tensor gradients update weights.

◆ Tensors are Important in Deep Learning

Deep learning models are essentially **mathematical functions** that transform input tensors into output tensors using **layers**.

- **Input Data Representation:**

- Images → 4D tensors (batch_size, height, width, channels).
- Text → 2D/3D tensors (batch_size, sequence_length, embedding_dim).
- Time series → 3D tensors (batch_size, timesteps, features).

- **Weights & Biases:**

- Each layer has trainable tensors for weights and biases.
- Example: Dense (fully connected) layer with 128 inputs and 64 outputs has a **weight tensor of shape [128, 64]**.

- **Forward Propagation:**

- Tensors flow through layers (matrix multiplication, activation functions).
- Example:
$$Y = f(X \cdot W + b)$$

X = input tensor,
W = weight tensor,
b = bias tensor.

- **Backpropagation (Training):**

- GradientTape computes **gradients of loss wrt tensors (weights & biases)**.
- Optimizers (SGD, Adam) update the tensors to minimize loss.

◆ Relevance of Tensors in Neural Networks

1. **Data Storage:**

- All input/output and intermediate results are stored as tensors.

2. **Matrix Computations:**

- Neural networks rely heavily on **matrix multiplications** (tensor × tensor).
- GPUs/TPUs optimize tensor operations massively for speed.

3. Broadcasting:

- Bias vectors applied across a batch of inputs.
- Example: Adding a 1D tensor (bias) to a 2D tensor (matrix of activations).

4. Automatic Differentiation:

- Gradients wrt tensor values are essential for training.

5. Real Applications:

- **Computer Vision:** Images as tensors (Convolutional Neural Networks).
- **NLP:** Sentences converted to tensor embeddings (Transformers).
- **Speech Recognition:** Audio signals represented as tensors.
- **Reinforcement Learning:** States, actions, and rewards stored as tensors.

◆ Example Flow in Deep Learning with Tensors

👉 Suppose we build a simple image classifier:

- Input: Batch of 32 images, each 28×28 grayscale → Tensor shape [32, 28, 28, 1].
- Layer 1: Convolution filter (3×3×1×32) → Extracts features.
- Layer 2: Dense Layer (weights [6272, 128]) → Flattened features to hidden neurons.
- Output: Dense Layer (weights [128, 10]) → Probability tensor for 10 classes (digits 0–9).

At each step:

- **Forward pass:** Tensors flow through layers.
- **Backward pass:** Tensor gradients update weights.

TensorFlow Applications for Deep Learning

- **TensorFlow** is an open-source deep learning framework developed by Google.
- It provides tools for **tensor computations**, **automatic differentiation**, and **GPU/TPU acceleration**.
- Core components:
 - **Tensors** → Multidimensional arrays (like NumPy arrays but optimized for GPU/TPU).
 - **Operations (Ops)** → Functions that manipulate tensors.
 - **Graph Execution** and **Eager Execution** → Modes of running code.
 - **GradientTape** → Automatic differentiation engine.

Consider the below application programs to understand important concepts of TensorFlow

Tensor Constants

Constants are fixed values in the computation graph.

Program Example:

```
import tensorflow as tf

# Create constant tensors
a = tf.constant(5)
b = tf.constant(3)

print("Tensor a:", a)
print("Tensor b:", b)
```

Explanation:

- `tf.constant(value)` creates an **immutable tensor**.
- `a` and `b` are **0-D tensors (scalars)**.

Tensor Arithmetic Operations

TensorFlow supports element-wise operations like addition, subtraction, multiplication, and division.

Program Example:

```
import tensorflow as tf

a = tf.constant(5)
b = tf.constant(3)
```



```
add = tf.add(a, b)
sub = tf.subtract(a, b)
mul = tf.multiply(a, b)
div = tf.divide(a, b)

print("a + b =", add.numpy())
print("a - b =", sub.numpy())
print("a * b =", mul.numpy())
print("a / b =", div.numpy())
```

Explanation:

- `tf.add, tf.subtract, tf.multiply, tf.divide` → perform basic math.
- `.numpy()` converts tensor result into NumPy value (works in **Eager Execution** mode).

Tensor Operations with Matrices

Tensors extend beyond scalars to **2D matrices** and **ND arrays**.

Program Example:

```
import tensorflow as tf

# Define matrices
A = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)
B = tf.constant([[5, 6], [7, 8]], dtype=tf.float32)

matmul = tf.matmul(A, B)      # Matrix multiplication
transpose = tf.transpose(A)   # Transpose

print("Matrix A:\n", A.numpy())
print("Matrix B:\n", B.numpy())
print("A x B:\n", matmul.numpy())
print("Transpose of A:\n", transpose.numpy())
```

Explanation:

- `tf.matmul(A, B)` → Performs **matrix multiplication**.
- `tf.transpose(A)` → Switches rows and columns.
- TensorFlow automatically handles **linear algebra operations** used in deep learning (like weights × inputs).

Broadcasting in TensorFlow

Broadcasting allows operations between tensors of different shapes.

Program Example:

```
import tensorflow as tf

C = tf.constant([1, 2, 3], dtype=tf.float32)
D = tf.constant(2.0)

broadcast_add = C + D
broadcast_mul = C * D

print("C:", C.numpy())
print("C + D:", broadcast_add.numpy())
print("C * D:", broadcast_mul.numpy())
```

Explanation:

- Tensor [1, 2, 3] and scalar 2.0 are **broadcasted**.
- Rule: TensorFlow expands dimensions automatically if compatible.
- Useful in **neural networks** when applying bias terms across batches.

Automatic Differentiation with GradientTape

Deep learning requires gradient computation for optimization.

Program Example:

```
import tensorflow as tf

x = tf.Variable(3.0)

with tf.GradientTape() as tape:
    y = x**2 + 2*x + 1    # Function y

grad = tape.gradient(y, x)

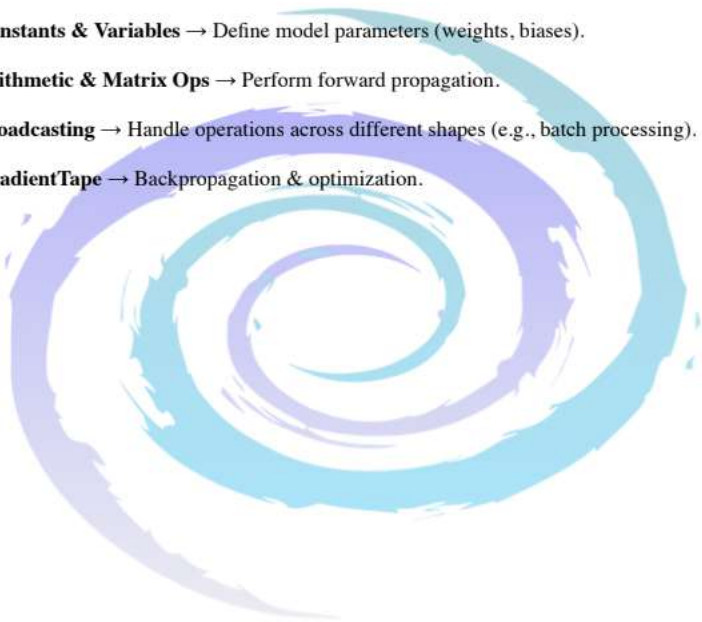
print("Function: y = x^2 + 2x + 1 at x=3")
print("Gradient dy/dx =", grad.numpy())    # Expected: 8
```

 Explanation:

- `tf.Variable` → trainable parameter.
- `tf.GradientTape` → records operations for automatic differentiation.
- `tape.gradient(y, x)` → computes derivative (dy/dx).
- Essential for **backpropagation** in neural networks.

Relevance to Deep Learning

- **Constants & Variables** → Define model parameters (weights, biases).
- **Arithmetic & Matrix Ops** → Perform forward propagation.
- **Broadcasting** → Handle operations across different shapes (e.g., batch processing).
- **GradientTape** → Backpropagation & optimization.



Mathematical Operations of Artificial Neural Network

Forward Propagation

- **Definition:**
Process of sending inputs through the network (layer by layer) to generate an output.
- **Explanation:**
Each layer transforms data → Output layer gives predictions.
- **Example:**
Image → [edges detected] → [digits recognized] → "Digit = 7".

Backward Propagation (Backpropagation)

- **Definition:**
Algorithm used to update weights by minimizing prediction error.
- **Explanation:**
 - Calculates error (Loss).
 - Finds gradient (slope) of error wrt weights.
 - Adjusts weights in opposite direction of gradient.
- **Example:**
If prediction = 0.9 but actual = 1, error is high → backprop adjusts weights to reduce error next time.

Gradient Descent

- **Definition:**
Optimization algorithm that reduces the loss function by updating weights gradually.
- **Explanation:**
Imagine descending a hill → we move step by step toward the lowest point (minimum loss).
- **Formula:**
$$w_{new} = w_{old} - \alpha \cdot (\partial w / \partial L)$$
- where
 α = learning rate.
- **Example:**
Suppose error = height of hill. The algorithm adjusts weights like moving downhill to reach the bottom (minimum error).

Learning Rate (α)

- **Definition:**
A parameter controlling how much weights are updated during training.
- **Explanation:**
 - Small $\alpha \rightarrow$ slow but stable learning.
 - Large $\alpha \rightarrow$ fast learning but may overshoot.
- **Example:**
If $\alpha = 0.01$, weights are updated in **tiny steps**. If $\alpha = 1.0$, steps are **huge**, risking instability.

Epoch

- **Definition:**
One full pass of the training dataset through the network.
- **Explanation:**
 - During each epoch, all data is seen once.
 - Multiple epochs are needed to train properly.
- **Example:**
Training on 1000 images for 10 epochs $\rightarrow$ Model sees **10,000 images in total** (same data, multiple passes).

Batch / Mini-batch

- **Definition:**
A subset of training data processed before updating weights.
- **Explanation:**
 - **Batch Gradient Descent:** Uses the whole dataset for one update.
 - **Mini-batch Gradient Descent:** Uses small groups (like 32 or 64 samples).
- **Example:**
If dataset = 10,000 images, batch size = 100 $\rightarrow$ 100 updates per epoch.

Important terms :

- **Neurons, layers, weights, and bias** form the **architecture** of ANN.
- **Forward propagation** $\rightarrow$ gives predictions.
- **Backward propagation + Gradient Descent** $\rightarrow$ improves accuracy.
- **Learning rate, epochs, batches** $\rightarrow$ control **how well & how fast the model learns**.

Components of Artificial Neural Network

Neuron / Node / Perceptron

- **Definition:**
The smallest unit of an Artificial Neural Network, inspired by biological brain neurons.
- **Explanation:**
Each neuron takes inputs, multiplies them with weights, adds a bias, applies an activation function, and produces an output.
- **Formula:**

$$y=f(\sum(x_i \cdot w_i)+b)$$

- where:
 x_i = input values
 w_i = weights
 b = bias
 f = activation function
- **Example :**
Suppose a student's marks in Maths = 80, Science = 70. The network predicts "Pass/Fail".
 - Input = [80, 70]
 - Weights = [0.5, 0.4], Bias = 10
 - Output = $(80 \times 0.5 + 70 \times 0.4 + 10) = 40 + 28 + 10 = 78 \rightarrow$ Passed.

Input Layer

- **Definition:**
The first layer of the network where raw data enters.
- **Explanation:**
Each node in this layer represents one feature from the dataset.
- **Example:**
In handwriting recognition (MNIST), each image (28x28 pixels) is flattened into **784 inputs**.

Hidden Layer

- **Definition:**
Layers between input and output layers where transformations occur.
- **Explanation:**
Hidden layers learn complex patterns by applying weights, bias, and nonlinear functions.
- **Example:**
In image classification, hidden layers detect **edges, shapes, and higher-level features** step by step.

Output Layer

- **Definition:**
The final layer that produces the network's predictions.
- **Explanation:**
Its size depends on the task:
 - Binary classification → 1 node (e.g., Yes/No).
 - Multi-class classification → N nodes (softmax gives probabilities).
- **Example:**
Predicting digits 0–9 → Output layer with **10 neurons**.

Weights

- **Definition:**
Numerical values representing the strength of connections between neurons.
- **Explanation:**
 - High weight → strong influence of input on output.
 - Low/zero weight → weak/no influence.
- **Example:**
In a spam filter, the word "free" may get a **high weight** (strong spam indicator).

Bias

- **Definition:**
An additional parameter that allows shifting the activation function.
- **Explanation:**
It helps the model fit data better, even if all inputs are zero.
- **Example:**
In linear regression y

$$y = mx + c$$

y=mx+c, the **bias** is like the intercept c

Activation Functions of Artificial Neural Network

Activation functions introduce **non-linearity** into the neural network, allowing it to learn **complex patterns**. Without them, the network would act like just a linear regression model.

Sigmoid / Logistic Function

- Definition:**

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Range:** $0 \rightarrow 1$

- Explanation:**

- Converts any input into a probability-like value.
- Commonly used in binary classification.

- Example:**

If input = 2 $\rightarrow$ sigmoid(2) $\approx$ 0.88 $\rightarrow$ interpreted as 88% probability of being "positive class."

- Limitation:** Can suffer from **vanishing gradients** for very large or very small values.

ReLU (Rectified Linear Unit)

- Definition:**

$$f(x) = \max(0, x)$$

- Range:** $0 \rightarrow \infty$

- Explanation:**

- If input $< 0 \rightarrow$ output = 0.
- If input $\geq 0 \rightarrow$ output = input itself.
- Makes networks train faster and deeper.

- Example:**

If inputs = [-2, 0, 3] $\rightarrow$ ReLU outputs [0, 0, 3].

- Limitation:** "Dead ReLU" problem when too many outputs become zero.

Tanh (Hyperbolic Tangent)

- **Definition:**

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- **Range:** $-1 \rightarrow 1$
- **Explanation:**
 - Like sigmoid, but centered at 0.
 - Useful when data has both positive and negative values.
- **Example:**
If input = 1 $\rightarrow \tanh(1) \approx 0.76$.
- **Advantage:** Stronger gradients than sigmoid, better for training deep networks.

Softmax

- **Definition:**

$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

- **Range:** $0 \rightarrow 1$, sum of all outputs = 1.
- **Explanation:**
 - Converts raw outputs into probability distribution.
 - Mainly used in the **output layer** for multi-class classification.
- **Example:**
Input scores = [2, 1, 0.1] $\rightarrow \text{Softmax} \approx [0.65, 0.24, 0.11]$.
(Class 1 has the highest probability).

Error & Loss Functions of Artificial Neural Network

Loss functions measure **how wrong** the network's predictions are. The training process tries to minimize this error.

Loss Function

- **Definition:** A mathematical function that measures the difference between predicted output and actual (true) output.

- **Types:**

- **Mean Squared Error (MSE):** For regression.

$$L = \frac{1}{n} \sum (y_{true} - y_{pred})^2$$

- **Cross-Entropy Loss:** For classification.

$$L = - \sum y_{true} \log(y_{pred})$$

- **Example:**

True = [0,1], Predicted = [0.2,0.8] → Cross-Entropy Loss is small (good prediction).

Cost Function

- **Definition:** The **average loss** over all training samples.
- **Explanation:**
 - Loss is calculated per sample, Cost = average over batch/epoch.
- **Example:**

If loss for 3 samples = [0.1, 0.2, 0.3], Cost = (0.1+0.2+0.3)/3 = 0.2.

Overfitting

- **Definition:** When ANN memorizes training data but fails to generalize to new data.
- **Symptoms:**
 - Very low training error, high test error.
- **Example:**

A student memorizing answers but failing in real exams.

Underfitting

- **Definition:** When ANN fails to capture underlying patterns in data.
- **Symptoms:**
 - High training error and high test error.
- **Example:**
A student not studying enough → poor performance everywhere.

Regularization Techniques

Used to prevent **overfitting** by reducing model complexity.

- **L1 Regularization (Lasso):**
Adds absolute values of weights as penalty.
Encourages **sparsity** (some weights become zero).
- **L2 Regularization (Ridge):**
Adds squared values of weights as penalty.
Keeps weights small, prevents large unstable updates.
- **Dropout:**
Randomly turns off some neurons during training.
Forces network to learn robust features instead of memorizing.
- **Example:**
In image classification, dropout ensures the model learns **general patterns** (like edges, shapes) instead of memorizing exact training images.

Important terms :

- **Activation Functions:** Decide how neurons fire (Sigmoid, ReLU, Tanh, Softmax).
- **Loss Functions:** Measure how wrong predictions are.
- **Overfitting/Underfitting:** Explain generalization performance.
- **Regularization:** Prevents overfitting and improves robustness.

Feedforward Neural Network (FNN)

A **Feedforward Neural Network (FNN)** is the simplest kind of neural network where information flows in one direction: **inputs** → **hidden layer(s)** → **output**.

There are **no loops** or feedback connections.

It maps an input vector to an output (a prediction) using layers of neurons that apply linear combinations and non-linear activations.

An FNN repeatedly applies:

Linear: $z = W \cdot a + b \rightarrow$ **Non-linear:** $a = g(z) \rightarrow$ next layer $\rightarrow$ final output $\rightarrow$ compute loss $\rightarrow$ adjust W, b by gradient descent.

Important terms

- **Neuron / Node:** A tiny function that takes numbers, multiplies by **weights (W)**, adds a **bias (b)**, applies an **activation**.
- **Layer:** A group of neurons.
 - **Input layer:** just the features (no computation).
 - **Hidden layer(s):** do most of the learning.
 - **Output layer:** produces the prediction (class scores or a number).
- **Weights (W):** Strength of connections; learned values.
- **Bias (b):** Constant added to each neuron; lets the neuron shift.
- **Activation (g):** Adds nonlinearity so the network can learn curved/complex patterns. Common:
 - $\text{ReLU}(x) = \max(0, x)$ (fast, popular)
 - $\text{sigmoid}(x) = 1 / (1 + e^{-x})$ (probabilities for binary)
 - $\tanh(x)$ (values in $-1..1$)
 - **softmax** (turns scores into class probabilities for multi-class)
- **Forward pass:** Compute outputs given inputs and current parameters.
- **Loss function:** How wrong we are.
 - Regression: **MSE**
 - Binary classification: **Binary Cross-Entropy (Log loss)**
 - Multi-class: **Cross-Entropy**

- **Backward pass (Backpropagation):** Compute gradients of loss w.r.t. parameters using the chain rule.
- **Gradient Descent:** Update rule to reduce loss.
 $W \leftarrow W - \alpha * (\partial L / \partial W)$; α is **learning rate**.
- **Epoch:** One full pass over the training set.
- **Batch size:** How many samples we use before one update (full-batch, mini-batch, or stochastic).
- **Hyperparameters:** Not learned—you choose them (learning rate, hidden units, layers, epochs, batch size, activation, optimizer, regularization).
- **Regularization:** Prevent overfitting (L2, dropout, early stopping).

Mathematical explanation

For layer ℓ :

- **Linear:** $z^{(\ell)} = W^{(\ell)} \cdot a^{(\ell-1)} + b^{(\ell)}$
- **Activation:** $a^{(\ell)} = g^{(\ell)}(z^{(\ell)})$
 with $a^{(0)} = x$ (input), and $a^{(L)}$ the network output.

Binary classification head (one output):

- $\hat{y} = \sigma(z) = 1 / (1 + e^{(-z)})$
- Loss (per sample): $L = -[y \log \hat{y} + (1-y) \log(1-\hat{y})]$
- Useful gradient: $\partial L / \partial z = \hat{y} - y$ (beautifully simple!)

Softmax head (multi-class):

- $\hat{y}_k = \exp(z_k) / \sum_j \exp(z_j)$
- Loss: cross-entropy $L = -\sum_k y_k \log(\hat{y}_k)$

Step-by-step numeric example (forward pass)

A $2 \rightarrow 2 \rightarrow 1$ network, ReLU hidden, sigmoid output.

Input: $x = [2.0, 3.0]$

Hidden layer (2 neurons):

Weights $W1 = [[0.5, -0.2], [0.8, 0.4]]$, Bias $b1 = [0.1, -0.1]$

- Neuron 1:
 $z1 = 0.5*2 + (-0.2)*3 + 0.1 = 1.0 - 0.6 + 0.1 = 0.5$
 $a1 = \text{ReLU}(0.5) = 0.5$
- Neuron 2:
 $z2 = 0.8*2 + 0.4*3 - 0.1 = 1.6 + 1.2 - 0.1 = 2.7$
 $a2 = \text{ReLU}(2.7) = 2.7$

Hidden output $\hat{a}(1) = [0.5, 2.7]$

Output layer (1 neuron, sigmoid):

Weights $W2 = [1.2, -0.7]$, Bias $b2 = 0.05$

$z_{\text{out}} = 1.2*0.5 + (-0.7)*2.7 + 0.05 = 0.6 - 1.89 + 0.05 = -1.24$

$\hat{y} = \text{sigmoid}(-1.24) \approx 0.2244$ ($\approx 22.44\%$ positive class probability)

This is exactly how a forward pass works, layer by layer.

Practical implementation of above example :

```
import math

def relu(x): return max(0.0, x)
def sigmoid(z): return 1/(1+math.exp(-z))

x = [2.0, 3.0]
W1 = [[0.5, -0.2],
      [0.8, 0.4]]
b1 = [0.1, -0.1]

# Hidden neuron 1
z1 = W1[0][0]*x[0] + W1[0][1]*x[1] + b1[0]
a1 = relu(z1)
print(f"Neuron1: z={z1:.3f}, a={a1:.3f}")

# Hidden neuron 2
z2 = W1[1][0]*x[0] + W1[1][1]*x[1] + b1[1]
a2 = relu(z2)
print(f"Neuron2: z={z2:.3f}, a={a2:.3f}")

# Output layer
W2 = [1.2, -0.7]; b2 = 0.05
z_out = W2[0]*a1 + W2[1]*a2 + b2
yhat = sigmoid(z_out)
print(f"Output: z={z_out:.3f}, sigmoid(y)={yhat:.4f}")
```

Example using SKlearn

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Synthetic 2D classification
X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                                     random_state=42)

# 2 hidden layers: 16 and 8 neurons, ReLU + Adam optimizer
clf = MLPClassifier(hidden_layer_sizes=(16,8), activation='relu',
                    solver='adam', max_iter=500, random_state=42)
clf.fit(Xtr, ytr)

print("Test accuracy:", accuracy_score(yte, clf.predict(Xte)))
```

- Same feedforward concept; library handles backprop/optimizers.
- You only choose architecture and hyperparameters.

When to use an FNN

- Tabular data (numerical/categorical features).
- Simple image/text problems as baselines

Common issues in FNN

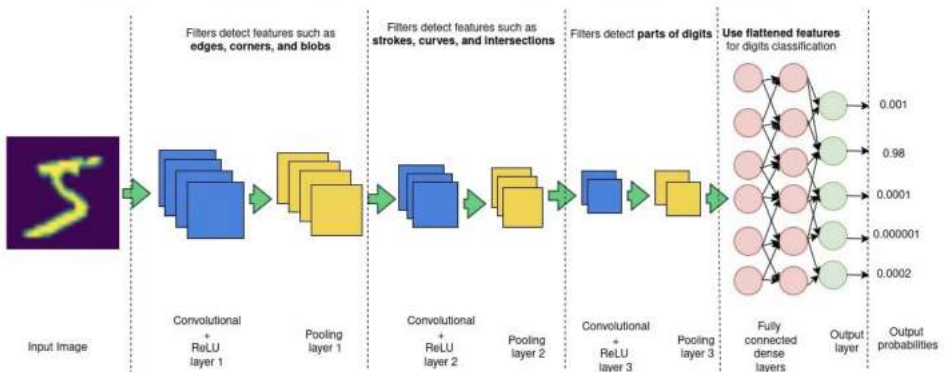
- **No nonlinearity** $\Rightarrow$ can't model XOR/complex patterns. Always use an activation in hidden layers.
- **Learning rate too high/low**: Too high diverges; too low is painfully slow.
- **Unscaled features**: Standardize inputs for stability.
- **Overfitting**: Use validation set, L2, dropout, early stopping.
- **Too shallow or too few neurons**: Underfits; add capacity gradually.

Why CNNs :

- A **Convolutional Neural Network (CNN)** is a type of deep learning model designed for processing **grid-like data** such as images.
- Instead of fully connecting every neuron to every pixel (like in Feedforward Neural Networks), CNNs use **convolutional layers** that apply small filters (kernels) across the image to detect **patterns (edges, shapes, textures)**.

- Images are large (e.g., $224 \times 224 \times 3 = 150k+$ values). Fully connected layers would be too big.
- CNNs exploit:
 - **Local connectivity** → look at small regions.
 - **Weight sharing** → same filter applied everywhere.
 - **Hierarchical learning** → low-level features (edges) → high-level features (objects).

Key Components of CNN



- Applies filters (small matrices like 3x3, 5x5) across the image.
- Each filter extracts specific features (edges, corners, textures).

👉 Example filter (edge detector):

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Pooling Layer

- Reduces the size of feature maps.
- **Max Pooling** (most common): takes the maximum value in a small window (e.g., 2×2).
- Helps with **dimensionality reduction** and prevents overfitting.

Output Layer

- Usually uses **Softmax** for classification.
- Example: For 10 classes → softmax gives 10 probabilities.

Input Image

- Images are stored as **pixel grids**.
- Grayscale → 2D matrix of values (0–255).
- RGB → 3D matrix (Height × Width × 3 channels).
Example: 224×224×3 for a color image.

Convolution

- The main operation in CNN.
- A **filter/kernel** (small matrix, e.g., 3×3) slides over the image.
- Computes a **dot product** → creates a **feature map**.
- Helps detect features like **edges, corners, textures**.

Formula:

Feature Map = Σ (Input × Kernel)

Kernel / Filter

- Small matrix (e.g., 3×3, 5×5) with weights.

- Shared across the image → fewer parameters than fully connected networks.
- Each kernel detects a **specific feature**.

Feature Map

- Output of a convolution operation.
- Shows **where the filter detected its pattern** in the image.

Stride

- Number of steps the filter moves when sliding.
- **Stride = 1** → moves 1 pixel at a time (large feature map).
- **Stride = 2** → moves 2 pixels at a time (smaller feature map).

Padding

- Adding extra pixels around the border of the image.
- Prevents shrinking of output after convolution.
- **Same padding** → output same size as input.
- **Valid padding** → no padding; output shrinks.

Channels

- **Depth** of the input image.
- Example: RGB = 3 channels.
- CNN learns **multiple filters per layer** (e.g., 32 filters → 32 feature maps).

Activation Function (ReLU)

- Introduces non-linearity.
- Most common: **ReLU** → replaces negative values with 0.

$$f(x) = \max(0, x)$$

Pooling

- Reduces size of feature maps → less computation, avoids overfitting.
- **Max Pooling**: takes max value in a region.

- **Average Pooling:** takes average.

Example (2×2 Max Pooling):

```
[1 3]      6  
[5 6]  →  (max value)
```

Flattening

- Converts 2D feature maps into a 1D vector.
- Required before connecting to fully connected (dense) layers.

Fully Connected Layer (Dense)

- Works like a regular neural network.
- Combines extracted features → predicts final class.

Softmax Layer

- Final layer for classification tasks.
- Converts raw scores into probabilities (sums to 1).
- Example: Cat → 0.85, Dog → 0.10, Car → 0.05.

Epoch

- One full pass of training data through the network.

Batch Size

- Number of samples processed before updating model weights.

Loss Function

- Measures error between predicted and actual labels.
- Common for CNN: **Cross-Entropy Loss**.

Optimizer

- Algorithm that updates weights to minimize loss.
- Examples: **SGD, Adam, RMSprop**.

Backpropagation

- Process of calculating gradients and updating weights using **chain rule**.
- CNNs use **backpropagation through convolution and pooling layers**.

Dropout

- Regularization technique → randomly drops neurons during training.
- Prevents overfitting.

Data Augmentation

- Artificially increasing dataset by applying transformations:
 - Rotation, flipping, zooming, cropping.
- Helps CNN generalize better.

Transfer Learning

- Using a pre-trained CNN (like VGG16, ResNet, MobileNet) and fine-tuning it for a new task.
- Saves training time and resources.

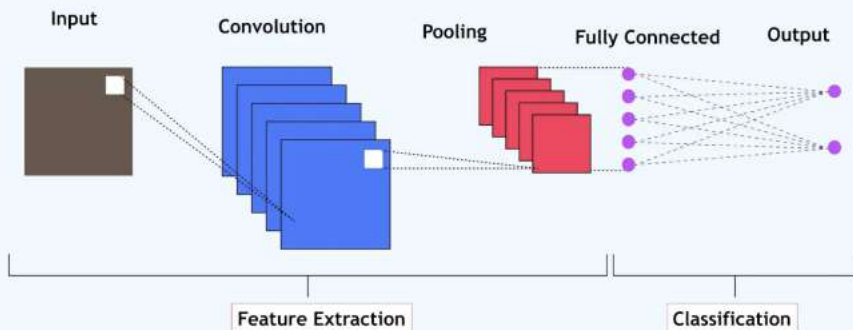
CNN Architectures

- Famous CNN models:
 - **LeNet-5** → First CNN (1998).
 - **AlexNet** → Won ImageNet 2012.
 - **VGGNet** → Deep, simple 3×3 filters.
 - **ResNet** → Introduced skip connections.
 - **MobileNet** → Lightweight for mobile/edge devices.

CNN Architecture (Typical Flow)

Input Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Fully Connected → Output

The Architecture of Convolutional Neural Networks



Deep Learning Project

Realtime Image Classification System using CNN

This project is a **real-time image classification system** using a **pre-trained Convolutional Neural Network (CNN)** (MobileNetV2) and our webcam.

- **Goal:** Capture live video from your webcam and classify each frame into one of the **ImageNet** categories (like dog, cat, keyboard, etc.).
- **Core Idea:** Use a **pre-trained CNN model (MobileNetV2)** to recognize objects without having to train from scratch.
- **Output:** Display the webcam feed with a label (class name + confidence percentage) overlaid in real-time.

Complete Source Code

```
import cv2
import numpy as np
from tensorflow.keras.applications.mobilenet_v2 import MobileNetV2, preprocess_input,
decode_predictions

def MarvellousImageClassifier():
    # 1) Load pre-trained CNN (ImageNet)
    model = MobileNetV2(weights="imagenet")

    # 2) Open webcam
    cap = cv2.VideoCapture(0)
    if not cap.isOpened():
        print("Error: Could not open webcam.")
        return

    while True:
        ret, frame = cap.read()
        if not ret:
            print("Error: Could not read frame.")
            break

        # 3) Preprocess for MobileNetV2: BGR -> RGB, resize to 224x224, scale
        img = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        img_resized = cv2.resize(img, (224, 224))
        x = np.expand_dims(img_resized, axis=0).astype(np.float32)
        x = preprocess_input(x)

        # 4) Predict
        preds = model.predict(x, verbose=0)
        decoded = decode_predictions(preds, top=1)[0][0] # (class_id, class_name, score)
        label = f"{decoded[1]}: {decoded[2]*100:.1f}%"

        # 5) Overlay prediction on the frame
        cv2.putText(frame, label, (16, 40), cv2.FONT_HERSHEY_SIMPLEX, 1.0, (0, 255, 0), 2, cv2.LINE_AA)

        cv2.imshow("Real-time CNN Classification (MobileNetV2)", frame)

        # 6) Exit
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
```

```
# 7) Cleanup
cap.release()
cv2.destroyAllWindows()

def main():
    MarvellousImageClassifier()

if __name__ == "__main__":
    main()
```

Explanation of project

1. Importing Libraries

```
import cv2
import numpy as np
from tensorflow.keras.applications.mobilenet_v2 import MobileNetV2,
preprocess_input, decode_predictions
```

- **cv2 (OpenCV):** For webcam capture, image processing, and display.
- **numpy:** For numerical operations (reshape, expand dimensions).
- **MobileNetV2 + utils:** Pre-trained CNN model + preprocessing + decoding predictions.

2. Loading the Pre-trained Model

```
model = MobileNetV2(weights="imagenet")
```

- Loads **MobileNetV2**, a **lightweight CNN** trained on **ImageNet (1.4M images, 1000 classes)**.
- Since it's pre-trained, you don't need to train it again — it's ready for inference.

3. Accessing the Webcam

```
cap = cv2.VideoCapture(0)
```

- Opens the **default webcam** (index 0).
- If webcam isn't found, it shows an error.

4. Frame Capture & Preprocessing

```
ret, frame = cap.read()
img = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
img_resized = cv2.resize(img, (224, 224))
x = np.expand_dims(img_resized, axis=0).astype(np.float32)
x = preprocess_input(x)
```

- **Frame:** Captures one video frame.
- **BGR → RGB:** OpenCV uses BGR, but MobileNet expects RGB.

- **Resize:** CNN input size = 224x224.
- **Expand dimensions:** CNN expects input shape (1, 224, 224, 3).
- **Preprocess:** Normalizes pixels for MobileNetV2.

5. Prediction

```
preds = model.predict(x, verbose=0)
decoded = decode_predictions(preds, top=1)[0][0]
label = f"{decoded[1]}: {decoded[2]*100:.1f}%"
```

- **model.predict:** Runs forward pass through CNN.
- **decode_predictions:** Converts raw model output into readable labels (e.g., "Labrador retriever").
- **Top-1 label:** Takes the best match with confidence percentage.

6. Overlay Prediction

```
cv2.putText(frame, label, (16, 40), cv2.FONT_HERSHEY_SIMPLEX, 1.0,
(0, 255, 0), 2, cv2.LINE_AA)
cv2.imshow("Real-time CNN Classification (MobileNetV2)", frame)
```

- Draws the prediction text on the video frame.
- Displays the live webcam feed with classification results.

7. Exit & Cleanup

```
if cv2.waitKey(1) & 0xFF == ord('q'):
    break
cap.release()
cv2.destroyAllWindows()
```

- Press 'q' to quit.
- Releases webcam and closes all windows.

Important Concepts

- **CNN (Convolutional Neural Network):** Learns spatial features from images (edges → textures → objects).
- **Transfer Learning:** Using a model trained on a huge dataset (ImageNet) for your own application.
- **MobileNetV2:** A lightweight CNN optimized for speed and accuracy, good for real-time applications.

OpenCV and CNN in Deep Learning

OpenCV :

- **OpenCV (Open Source Computer Vision Library)** is a powerful library for image processing and computer vision.
- Written in C++ with Python bindings.
- Used in:
 - Image transformations (resize, rotate, filter).
 - Feature detection (edges, corners, faces).
 - Video analysis.
 - Object recognition (integrated with CNN/DL models).

Key Concepts

- **Image Representation:** Stored as a matrix (pixels). Each pixel has values for **BGR** (Blue, Green, Red) in OpenCV.
- **Grayscale Conversion:** Simplifies image → single intensity value (0–255).
- **Edge Detection:** Finds sudden intensity changes (object boundaries).
- **Integration with Deep Learning:** OpenCV provides pre-processing before sending images to CNNs.

CNN (Convolutional Neural Network)

- **CNN** is a type of deep learning model specialized for images.
- Learns features **automatically** (edges → shapes → objects).
- Key layers:
 - **Convolution Layer:** Extracts features using filters.
 - **Pooling Layer:** Reduces size, keeps important features.
 - **Fully Connected Layer:** Classifies objects.
- Used in:
 - Image classification.
 - Object detection.
 - Face recognition.

Code Explanations

OpenCVPixel.py

Concept: Show image pixel values.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Read image
img = cv2.imread("sample.jpg")

# 2. Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 3. Display image + pixel grid
plt.figure(figsize=(10,5))
plt.subplot(1,2,1); plt.imshow(gray, cmap="gray");
plt.title("Grayscale Image"); plt.axis("off")
plt.subplot(1,2,2); plt.imshow(gray, cmap="gray");
plt.colorbar(label="Pixel Value"); plt.title("Pixel Values")
plt.show()
```

Learning:

- How images are stored as pixel grids.
- Visualization of intensity values.

Edge.py

Concept: Detect edges in an image.

```
import cv2

# Read image
img = cv2.imread("sample.jpg", 0) # grayscale

# Canny Edge Detection
edges = cv2.Canny(img, 100, 200)

cv2.imshow("Original", img)
cv2.imshow("Edges", edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Learning:

- Edge detection helps CNN identify **boundaries of objects**.
- OpenCV's **Canny** algorithm works in 3 steps:
 1. Noise reduction.
 2. Gradient calculation.
 3. Detecting edges.

CNNedgeDetection.py

Concept: Using CNN for edge detection/classification.

```
import cv2
import numpy as np
from tensorflow.keras.applications import MobileNetV2,
preprocess_input, decode_predictions

# Load pre-trained CNN
model = MobileNetV2(weights="imagenet")

# Capture from webcam
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    img = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    img_resized = cv2.resize(img, (224, 224))
    x = np.expand_dims(img_resized, axis=0).astype(np.float32)
    x = preprocess_input(x)

    # Prediction
    preds = model.predict(x, verbose=0)
    decoded = decode_predictions(preds, top=1)[0][0]
    label = f"{decoded[1]}: {decoded[2]*100:.1f}%"

    # Show on screen
    cv2.putText(frame, label, (16, 40), cv2.FONT_HERSHEY_SIMPLEX, 1,
    (0,255,0), 2)
    cv2.imshow("CNN Classification", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Learning:

- Webcam captures **real-time frames**.
- Frames → CNN (MobileNetV2) → Predict object.
- CNN automatically detects **edges** → **shapes** → **object label**.



Recurrent Neural Networks (RNN)

- **Recurrent Neural Networks (RNNs)** are a class of neural networks designed for **sequential data** (text, speech, time-series, video frames).
- Unlike **Feedforward Neural Networks (FNNs)** and **Convolutional Neural Networks (CNNs)**, RNNs have a **memory mechanism** that helps them remember **previous inputs**.
- They are widely used in **Natural Language Processing (NLP)**, **speech recognition**, **music generation**, and **time-series forecasting**.

Sequence Data

- Data that has **order** and **context matters**.
- Example: A sentence $\rightarrow$ "I love AI" $\rightarrow$ the word "love" depends on "I".

Hidden State

- At each time step, the RNN maintains a **hidden state (h_t)**.
- This hidden state captures **information from previous inputs**.
- Formula:

$$h_t = f(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

where:

- h_t = hidden state at time t
- x_t = input at time t
- W_h, W_x = weight matrices
- f = activation function (usually tanh/ReLU)

Timestep

- One **unit of time** in sequence processing.
- Example: For text input, each word is a timestep.

Input Feature

- The dimension of each input.
- Example: Word embeddings (vector representation of words) $\rightarrow$ each word is represented as a vector of numbers.

Output

- RNN can produce:
 - **Many-to-One:** Sentiment analysis (entire sentence → one label).
 - **Many-to-Many:** Language translation (sequence → sequence).

Variants of RNN

- **Vanilla RNN** → basic RNN, suffers from vanishing gradient problem.
- **LSTM (Long Short-Term Memory)** → solves long-term dependency issue with gates (input, forget, output).
- **GRU (Gated Recurrent Unit)** → simpler than LSTM but effective.

Uses in Development

- **NLP:** Sentiment analysis, machine translation, text generation.
- **Speech Recognition:** Convert audio sequence to text.
- **Time-Series Forecasting:** Predict stock prices, weather data.
- **Music Generation:** Learn sequences of notes.

Simple Python Code Example

Here's a **basic RNN example with Keras** to demonstrate sequential input:

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense, Embedding

# Sample: Sequence classification
# Suppose we have 10 sequences, each with 5 timesteps, vocabulary
size = 20
X = np.random.randint(20, size=(10, 5)) # Input sequences (10
samples, 5 timesteps)
y = np.random.randint(2, size=(10, 1)) # Binary output (labels)

# Build RNN model
model = Sequential()
model.add(Embedding(input_dim=20, output_dim=8, input_length=5)) #
Word embedding
model.add(SimpleRNN(16, activation='tanh')) #
RNN layer
model.add(Dense(1, activation='sigmoid')) #
Output layer

# Compile model
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

# Train
model.fit(X, y, epochs=5, verbose=1)

# Prediction
print("Sample Prediction:", model.predict(X[:1]))
```

Explanation of Code

1. **Embedding Layer:** Converts integer-encoded words → dense vectors.
2. **SimpleRNN Layer:** Processes sequences one timestep at a time, remembers past states.
3. **Dense Layer:** Outputs final classification.
4. **Prediction:** Outputs probability (0–1) for binary class.

Sequence Data :

- **Sequence Data** = Data where **order matters**.
- Each element depends on the **previous element(s)**.
- Examples:
 - 📖 Text: A sentence → meaning depends on word order.
 - "I love AI" ≠ "AI love I"
 - 🎵 Audio: A speech signal → each sound depends on previous sounds.
 - 📈 Time-series: Stock price today depends on yesterday's price.

In **normal feedforward neural networks**, inputs are **independent**. But in **sequence tasks**, we need **context**.

How RNN Handles Sequence Data

Step-by-Step Process

1. Input sequence is fed **one element at a time** (word by word, timestep by timestep).
2. At each timestep **t**, RNN takes:
 - Current input (x_t)
 - Previous hidden state (h_{t-1}) (memory from past)
 - Combines them to compute **new hidden state** (h_t)
3. Hidden state (h_t) = acts like **short-term memory**.
4. At the end, RNN can output:
 - Final result (e.g., sentiment of whole sentence)
 - Or sequence of outputs (e.g., translation word-by-word).

Memory Mechanism in RNN

- **Hidden State** = Memory cell that carries **past information**.

- Formula:

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

- where:

- x_t = current input
- h_{t-1} = previous memory
- h_t = new memory
- f = activation function (tanh, ReLU)

Example

Sentence: **"I love AI"**

- At timestep 1: input = "I", memory starts forming.
- At timestep 2: input = "love", RNN remembers "I" + "love".
- At timestep 3: input = "AI", RNN remembers "I love AI".

Thus, RNN builds meaning step-by-step.

Simple Visualization

$x_1 \rightarrow [\text{RNN Cell}] \rightarrow h_1$
 $x_2 \rightarrow [\text{RNN Cell}] \rightarrow h_2$
 $x_3 \rightarrow [\text{RNN Cell}] \rightarrow h_3 \rightarrow \text{output}$

- Each **RNN cell** passes **memory (h)** forward.
- Like a **chain** of connected cells.

Small Python Example to Demonstrate Memory

Here's a **toy RNN implementation** to show how it processes a sequence:

```
import numpy as np

# Example sentence: "I love AI" (encoded as numbers)
sequence = [1, 2, 3] # Let's say 1=I, 2=love, 3=AI

# Initialize weights and hidden state
Wx, Wh, b = 0.5, 0.8, 0.1 # random chosen values
h = 0 # initial hidden state

print("Processing sequence step by step:")

for t, x in enumerate(sequence):
    h = np.tanh(Wx * x + Wh * h + b) # memory update
    print(f"TimeStep {t+1} | Input={x} | Hidden State={h:.4f}")
```

Output Explanation

- At **timeStep 1**, RNN only sees "I" → stores memory.
- At **timeStep 2**, it sees "love" + remembers "I".
- At **timeStep 3**, it sees "AI" + remembers "I love".

This demonstrates **sequence + memory flow**.

Why This Matters in Development

- RNNs are powerful because they **don't forget the past immediately**.
- Example applications:
 - **Chatbot** → remembers previous words in conversation.
 - **Stock predictor** → remembers yesterday's price.
 - **Music generator** → remembers last few notes.

Advantages & Limitations

Advantages:

- Good for sequential data.
- Captures context (history).

Limitations:

- **Vanishing Gradient** → hard to remember long-term dependencies.
- Slow for very long sequences.
- Replaced in modern NLP by **Transformers** (e.g., **BERT, GPT, ChatGPT**).

Summary

- RNNs are neural networks for **sequential tasks**.
- They maintain **memory** using hidden states.
- Widely used in **NLP, speech, and forecasting**.
- Advanced versions: **LSTM & GRU**.
- Modern replacements: **Transformers**.

Relationship Between Deep Learning, RNN, NLP, Transformers, LLM & GenAI

1. Deep Learning – The Foundation

- **Definition:** Subfield of Machine Learning using **neural networks with multiple layers**.
- **Goal:** Learn **complex patterns** from large amounts of data (images, text, audio).
- **Architectures in Deep Learning:**
 - **FNN (Feedforward NN)** → Basic networks.
 - **CNN (Convolutional NN)** → Best for images.
 - **RNN (Recurrent NN)** → Best for sequences (text, speech, time-series).
 - **Transformers** → Best for long sequences, modern NLP.

Deep Learning = Parent field. Everything else (RNN, Transformers, LLM, GenAI) grows from here.

2. NLP – Application Area of Deep Learning

- **Definition:** Natural Language Processing = Teaching machines to **understand and generate human language**.
- **Why Deep Learning?**
 - Classical NLP (rule-based, statistical) struggled with complexity.
 - Deep Learning allowed **automatic feature extraction** + handling of complex language structures.

NLP = Problem Domain (language).

Deep Learning = Tools (RNN, Transformers) used to solve NLP problems.

3. RNN – First Breakthrough for NLP

- **RNN (Recurrent Neural Networks):**
 - Designed to handle **sequence data** (sentences, speech).
 - Maintains **memory (hidden state)** → understands context of words.

- Variants: **LSTM, GRU** → better long-term memory.

- **Applications in NLP:**

- Sentiment analysis
- Machine translation (before Transformers)
- Speech recognition

Mapping:

NLP tasks → handled by RNN (first DL success in language).

But: RNNs struggled with **long texts** (vanishing gradient).

4. Transformers – Game Changer

- Introduced in 2017 (*Attention is All You Need*).
- Use **Self-Attention** → every word can look at every other word in sentence → captures long-range dependencies.
- **Advantages over RNN:**
 - Parallel processing (faster training).
 - Can handle long sequences without memory loss.
- **Applications:**
 - Translation (Google Translate now uses Transformers).
 - Summarization, Q&A, text classification.

Mapping:

Transformers replaced RNNs in NLP → became the **new standard**.

5. LLM – Large Language Models

- **Definition:** A Transformer model trained on **massive datasets** (billions/trillions of words).
- **Examples:** GPT (OpenAI), BERT, PaLM, LLaMA.
- **Characteristics:**
 - Learns grammar, facts, reasoning.
 - Can **generate coherent, human-like text**.
- **Why “Large”?**
 - Billions of parameters → store knowledge about world.

Mapping:

LLMs = **Large-scale implementation of Transformers** applied to NLP.

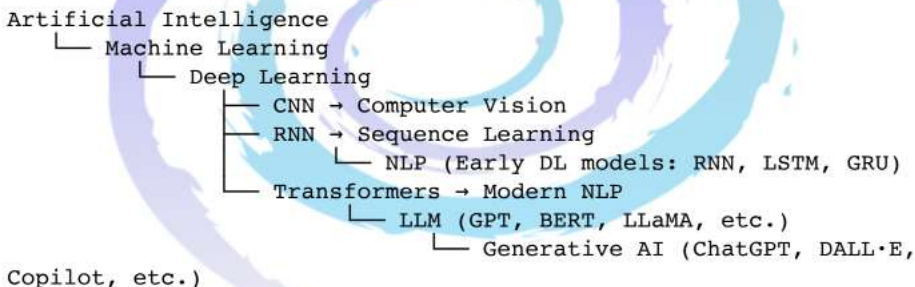
6. Generative AI (GenAI)

- **Definition:** AI that can **generate new content** (text, images, music, code).
- **LLMs in GenAI:**
 - ChatGPT (text generation)
 - Code assistants (Copilot)
 - Multi-modal models (text → image, text → video)
- **Beyond text:**
 - Diffusion Models → Images (DALL·E, Stable Diffusion)
 - Audio generation (MusicLM)

Mapping:

GenAI = **Application layer** that uses **LLMs (text)**, **Diffusion models (images)**, **GANs/VAEs (music/video)**.

Full Hierarchical Flow



8. Simple Analogy that we discussed in session

- **Deep Learning** = Toolbox.
- **RNN** = Old hammer for sequences (good, but limited).
- **Transformers** = New electric drill (powerful & scalable).
- **LLM** = A mega machine built using Transformers.
- **GenAI** = Products we create using that mega machine (ChatGPT, DALL·E).

Summary Mapping

Concept	Role	Example
Deep Learning	Parent field, provides architectures	ANN, CNN, RNN, Transformers
NLP	Application domain (language)	Translation, Chatbots
RNN	Early DL architecture for NLP	Sentiment analysis, speech recognition
Transformers	Modern DL architecture for NLP	BERT, GPT
LLM	Large-scale Transformer trained on huge text	GPT-4, LLaMA
GenAI	AI that generates new content	ChatGPT, Copilot, DALL-E



RNN Character Prediction Application

```
import numpy as np

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import SimpleRNN, Dense

from tensorflow.keras.utils import to_categorical


def MarvellousCharcterPredictions():

    # 1) Prepare toy text dataset

    text = "hellohellohellohello" # repeating pattern

    chars = sorted(list(set(text))) # unique characters

    char_to_int = {c:i for i,c in enumerate(chars)}

    int_to_char = {i:c for i,c in enumerate(chars)}

    print("Unique characters:", chars)


    # 2) Convert text to integer sequence

    encoded = [char_to_int[c] for c in text]


    # 3) Create input-output pairs (sequence length = 3)

    seq_length = 3

    X, y = [], []

    for i in range(len(encoded) - seq_length):

        X.append(encoded[i:i+seq_length]) # e.g. "hel"

        y.append(encoded[i+seq_length]) # next char e.g. "l"


    X = np.array(X)

    y = np.array(y)


    # One-hot encode output
```

```
y = to_categorical(y, num_classes=len(chars))

# Reshape X for RNN (samples, timesteps, features)
X = X.reshape((X.shape[0], X.shape[1], 1))

# 4) Build RNN model
model = Sequential()
model.add(SimpleRNN(16, activation="tanh", input_shape=(seq_length, 1)))
model.add(Dense(len(chars), activation="softmax"))

model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])

# 5) Train
model.fit(X, y, epochs=300, verbose=0)

# 6) Test prediction
test_input = ["h", "e", "l"] # we expect model to predict "l"
encoded_input = np.array([[char_to_int[c] for c in test_input]]).reshape((1, seq_length, 1))

pred = model.predict(encoded_input, verbose=0)
predicted_char = int_to_char[np.argmax(pred)]

print("Input:", test_input, "-> Predicted next char:", predicted_char)

def main():
    MarvellousCharcterPredictions()

if __name__ == "__main__":
    main()
```


Explanation of above application

- **Task:** Next-character prediction.
- **Granularity:** Character level.
- **Model:** A simple recurrent neural network (SimpleRNN) with one hidden layer (16 units) + a softmax output over the unique characters in the text.

Data preparation

Raw text and vocabulary

```
text = "hellohellohellohello"
chars = sorted(list(set(text))) # unique characters
char_to_int = {c:i for i,c in enumerate(chars)}
int_to_char = {i:c for i,c in enumerate(chars)}
```

- text length = 20 characters ("hello" × 4).
- Unique characters in "hello" = {h, e, l, o} → after sorted(...): ['e', 'h', 'l', 'o'].
- Example mapping (because of alphabetical sort):
 - e→0, h→1, l→2, o→3
(Important: mapping depends on the **sorted order**, not the order of first appearance.)

Integer encoding the text

```
encoded = [char_to_int[c] for c in text]
```

- Converts each char to its integer id.
- This is **not** one-hot; it's just integer indices (inputs will be reshaped to numeric features of size 1; Keras casts to float internally).

Sliding window to make (X,y) pairs

```
seq_length = 3
X, y = [], []
for i in range(len(encoded) - seq_length):
    X.append(encoded[i:i+seq_length]) # 3 chars as input
    y.append(encoded[i+seq_length])   # 4th char as label
```

- With length 20 and seq_length=3, you get **17** training samples (from positions 0..16).
- A few concrete examples from the actual text:
 - indices 0-2: **h e l** → next is **l**

- indices 1-3: **e h l** → next is **o**
- indices 2-4: **h l o** → next is **h**
- indices 3-5: **l o h** → next is **e**
- ...and so on (a repeating cycle from the “hello” pattern).

One-hot encode the targets

```
y = to_categorical(y, num_classes=len(chars))
```

- `len(chars)=4`, so `y` becomes shape **(17, 4)**.
- Reason: we are doing **multi-class classification** (which of the 4 characters comes next?), so the label must be a one-hot vector.

Reshape inputs for RNN

```
X = np.array(X) # (17, 3)
X = X.reshape((X.shape[0], X.shape[1], 1)) # (samples, timesteps,
features) → (17, 3, 1)
```

- **Timesteps = 3** (we feed 3 characters in order).
- **Features = 1** (each timestep is a single numeric feature: the integer id).
 Note: In larger setups, we would typically use an **Embedding** or **one-hot inputs** instead of raw integers to avoid implying any “ordinal” meaning to ids.

Model architecture

```
model = Sequential()
model.add(SimpleRNN(16, activation="tanh", input_shape=(seq_length,
1)))
model.add(Dense(len(chars), activation="softmax"))
SimpleRNN(16, activation="tanh")
```

- **Recurrent layer** with 16 hidden units.
- Processes the sequence (3 timesteps) and returns the **last hidden state** (a vector of length 16).
- **Why tanh?** Values range in $[-1, 1]$, which helps stabilize recurrent dynamics. (ReLU is less common for vanilla RNNs due to exploding activations; LSTM/GRU are more robust alternatives.)

Parameter count (good to teach!):

- For SimpleRNN with U units and input dim D :
 - $\text{Weights} = U * (U + D + 1)$ (kernel $D \times U$, recurrent $U \times U$, bias U)

- Here $U=16, D=1 \rightarrow 16 * (16+1+1) = 16 * 18 = 288$ parameters.

Dense(4, activation="softmax")

- Maps the 16-dim hidden state to 4 logits (one per character).
- Softmax** converts logits to a probability distribution over the 4 characters.

Parameter count:

- Dense weights: $16 * 4 = 64$
- Bias: 4
- Total Dense params = **68**

Grand total = 288 (RNN) + 68 (Dense) = **356 parameters**.

Loss, optimizer, metrics

```
model.compile(loss="categorical_crossentropy", optimizer="adam",
metrics=["accuracy"])
```

- Loss:** `categorical_crossentropy` is the standard for one-hot multi-class classification. If $\mathbf{p}$ is the predicted probability vector and $\mathbf{y}$ is one-hot, the loss simplifies to $-\log(p_{\text{trueClass}})$.
- Optimizer:** `adam` is a robust default that adapts learning rates per parameter.
- Metric:** `accuracy` counts correct argmax predictions.

Training

```
model.fit(X, y, epochs=300, verbose=0)
```

- Trains on the 17 examples for **300 epochs**.
- With such a tiny dataset, the model will **overfit** (which is totally fine for a didactic demo—our goal is to make it memorize the pattern).
- No validation split is used here (again fine for a toy example).

Inference (predicting the next character)

```
test_input = ["h", "e", "l"] # we expect "l" next in the
pattern
encoded_input = np.array([[char_to_int[c] for c in
test_input]]).reshape((1, seq_length, 1))
```

```
pred = model.predict(encoded_input, verbose=0) # shape (1, 4)
predicted_char = int_to_char[np.argmax(pred)]
```

- `encoded_input` shape is **(1,3,1)** → one sample, 3 timesteps, 1 feature each.
- `pred` is a **(1,4)** probability vector.
- `np.argmax(pred)` gives the predicted class id; map back via `int_to_char`.
- For “**h e l**”, the next character in the repeating pattern is indeed “**l**”.



Sentiment Analysis with a SimpleRNN

```
import numpy as np

from tensorflow.keras.preprocessing.text import Tokenizer

from tensorflow.keras.preprocessing.sequence import pad_sequences

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Embedding, SimpleRNN, Dense
```

```
def MarvellousSentimentAnalysis():

    # 1) Tiny dataset (Positive = 1, Negative = 0)
    sentences = [

        "I love this movie",
        "This film was great",
        "What a fantastic experience",
        "I really enjoyed it",
        "Absolutely wonderful acting",
        "I hate this movie",
        "This film was terrible",
        "What a bad experience",
        "I really disliked it",
        "Absolutely horrible acting"

    ]

    print("Input dataset : ")
    print(sentences)

    labels = [1,1,1,1,1, 0,0,0,0,0] # 1=Positive, 0=Negative

    # 2) Tokenize text → convert words to integers
    tokenizer = Tokenizer(num_words=50) # keep top 50 words
    tokenizer.fit_on_texts(sentences)
    X = tokenizer.texts_to_sequences(sentences)
```



```
print("Word Index:", tokenizer.word_index) # show vocabulary

# 3) Pad sequences → same length input
maxlen = 5
X = pad_sequences(X, maxlen=maxlen)
y = np.array(labels)

# 4) Build simple RNN model
model = Sequential()

model.add(Embedding(input_dim=50, output_dim=8, input_length=maxlen)) # word
embeddings

model.add(SimpleRNN(8, activation="tanh"))
model.add(Dense(1, activation="sigmoid")) # binary output
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])

# 5) Train
model.fit(X, y, epochs=30, verbose=0)

# 6) Test on new examples
test_sentences = ["I enjoyed this film", "I hated this film"]
test_seq = tokenizer.texts_to_sequences(test_sentences)
test_seq = pad_sequences(test_seq, maxlen=maxlen)
pred = model.predict(test_seq)

for s, p in zip(test_sentences, pred):
    print(f"Sentence: {s} -> Sentiment: ", "Positive" if p>0.5 else "Negative")

def main():
    MarvellousSentimentAnalysis()

if __name__ == "__main__":
    main()
```


Explanation of above application

A tiny **binary sentiment classifier** (Positive = 1, Negative = 0) that reads a short sentence and predicts its sentiment using an **Embedding** → **SimpleRNN** → **Dense(sigmoid)** network.

Dataset & Labels

```
sentences = [
    "I love this movie", "This film was great",
    "What a fantastic experience", "I really enjoyed it",
    "Absolutely wonderful acting",
    "I hate this movie", "This film was terrible",
    "What a bad experience", "I really disliked it",
    "Absolutely horrible acting"
]
labels = [1,1,1,1,1, 0,0,0,0,0]
```

- 10 short sentences (5 positive, 5 negative).
- Balanced labels → avoids class-imbalance issues for this demo.

Tokenization (Text → Integer Sequences)

```
tokenizer = Tokenizer(num_words=50) # keep top 50 words (by frequency)
tokenizer.fit_on_texts(sentences)
X = tokenizer.texts_to_sequences(sentences)
print("Word Index:", tokenizer.word_index)
```

- **Tokenizer** builds a vocabulary from the training texts.
- `num_words=50` means: at model time we will keep only the 50 most frequent tokens (IDs 1..49 effectively; 0 is reserved for padding). Tokens with IDs ≥ 50 are treated as out-of-vocabulary if you later set an `oov_token`.
- `texts_to_sequences` maps each word to its integer ID.

Example: If `tokenizer.word_index` contains `{"i":1, "this":2, "film":3, "was":4, "great":5, ...}`, then

- "I love this movie" → [1, <id(love)>, 2, <id(movie)>].

Padding (Uniform Sequence Length)

`maxlen = 5`

```
X = pad_sequences(X, maxlen=maxlen)
y = np.array(labels)
```

- RNNs expect **equal-length** sequences. `pad_sequences` left-pads with zeros to make each sequence length = `maxlen` (here 5).
- Shape after padding: `X.shape == (10, 5)` (10 sentences, 5 tokens each). Zeros are **padding token**.

Why 5? It's arbitrary and small for demo. Longer sentences get **truncated** (from the start by default). You can control truncation/padding side with `padding='post'`, `truncating='post'`.

Model Architecture

```
model = Sequential()
model.add(Embedding(input_dim=50, output_dim=8, input_length=maxlen))
model.add(SimpleRNN(8, activation="tanh"))
model.add(Dense(1, activation="sigmoid"))
```

Embedding layer

- Purpose:** Learn a dense vector for each word ID (distributional semantics).
- Shapes:**
 - Input: `(batch, timesteps=5)` of integer IDs
 - Output: `(batch, timesteps=5, embed_dim=8)` (each token $\rightarrow$ 8-D vector)
- Parameters:** `input_dim * output_dim = 50 * 8 = 400` trainable params.

Important: `input_dim` must be $\geq$ the maximum token index your tokenizer will produce (bounded by `num_words`). With `num_words=50`, `input_dim=50` is consistent.

SimpleRNN layer

- Purpose:** Read the sequence of embeddings and produce a final hidden state summarizing the sentence.
- Config:** `SimpleRNN(8, activation="tanh")`
- Input dim D = 8** (embedding size), **units U = 8**.
- Output shape:** `(batch, 8)` (last hidden state).
- Parameters:** $U * (U + D + 1)$
 $\rightarrow 8 * (8 + 8 + 1) = 8 * 17 = 136$

Why `tanh`? Classic for vanilla RNNs. For more complex text, prefer **GRU/LSTM** to handle longer dependencies.

Dense output layer

- **Purpose:** Binary classification (positive vs negative).
- **Activation:** sigmoid $\rightarrow$ outputs probability p
- **Params:** weights $8 \times 1 = 8$ + bias $1 \rightarrow 9$.

Total params: 400 (Embedding) + 136 (RNN) + 9 (Dense) = **545**.

Compile & Train

```
model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.fit(X, y, epochs=30, verbose=0)
```

- **Loss:** `binary_crossentropy` fits a sigmoid output for binary labels.
- **Optimizer:** Adam is a solid default.
- **Metrics:** Accuracy for quick feedback.

With such a tiny dataset and 30 epochs, the model will likely **memorize**.

Inference (Predict on New Sentences)

```
test_sentences = ["I enjoyed this film", "I hated this film"]
test_seq = tokenizer.texts_to_sequences(test_sentences)
test_seq = pad_sequences(test_seq, maxlen=maxlen)
pred = model.predict(test_seq)
for s, p in zip(test_sentences, pred):
    print(f"Sentence: {s} -> Sentiment:", "Positive" if p>0.5 else
          "Negative")
```

- The **same tokenizer** is used at inference to ensure word-to-ID mapping consistency.
- Pad to `maxlen=5`.
- Apply **sigmoid** scores and threshold at **0.5**:
 - $p > 0.5 \rightarrow$ Positive; else Negative.

End-to-End Shapes

- **Training:**
 - $X: (10, 5) \rightarrow$ Embedding $\rightarrow (10, 5, 8) \rightarrow$ SimpleRNN $\rightarrow (10, 8) \rightarrow$ Dense $\rightarrow (10, 1)$

- **Inference:**

- test_seq: (2, 5) $\rightarrow$ (2, 5, 8) $\rightarrow$ (2, 8) $\rightarrow$ (2, 1)



Time-Series Next-Value Prediction with a SimpleRNN

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import SimpleRNN, Dense


# 1) Reproducibility

np.random.seed(42)

tf.random.set_seed(42)


# 2) Create a longer sequence (0..199)

data = np.arange(0, 200, dtype=np.float32)


# 3) Normalize to [0, 1] to help training

max_val = data.max()

data_norm = data / max_val


# 4) Make sliding windows: use 'window' numbers to predict the next one

window = 5 # timesteps

X, y = [], []

for i in range(len(data_norm) - window):

    X.append(data_norm[i:i+window])    # e.g. [0,1,2,3,4] (normalized)

    y.append(data_norm[i+window])      # next number (normalized)

X = np.array(X)[..., np.newaxis]      # shape: (samples, timesteps, features=1)

y = np.array(y)


print("X shape:", X.shape) # (samples, 5, 1)


# 5) Train / test split

split = int(0.8 * len(X))
```



```
X_train, X_test = X[:split], X[split:]
```

```
y_train, y_test = y[:split], y[split:]
```

```
# 6) Build a tiny RNN
```

```
model = Sequential([
```

```
    SimpleRNN(32, activation="tanh", input_shape=(window, 1)),
```

```
    Dense(1)
```

```
])
```

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.005),
```

```
              loss="mse")
```

```
# 7) Train
```

```
history = model.fit(X_train, y_train,
```

```
                    epochs=300, batch_size=32,
```

```
                    verbose=0,
```

```
                    validation_data=(X_test, y_test))
```

```
# 8) Helper to predict next number for a given (unnormalized) sequence
```

```
def predict_next(seq):
```

```
    seq = np.asarray(seq, dtype=np.float32)
```

```
    assert len(seq) == window, f"Need {window} numbers"
```

```
    seq_norm = (seq / max_val).reshape(1, window, 1)
```

```
    pred_norm = model.predict(seq_norm, verbose=0)[0, 0]
```

```
    return pred_norm * max_val # de-normalize back
```

```
# 9) Demo predictions
```

```
tests = [
```

```
    [95, 96, 97, 98, 99],    # expect ~100
```

```
    [7, 8, 9, 10, 11],      # expect ~12
```

```
    [100, 101, 102, 103, 104] # expect ~105
```



```
]
```

for t in tests:

```
    print(f"Input: {t} -> Predicted next: {round(predict_next(t))}")
```



Explanation of above application

- **Task:** Given the last **5 numbers** in a numeric sequence, predict the **next** number.
- **Data:** A simple increasing sequence 0, 1, 2, ..., 199.
- **Model:** A tiny RNN **regression** model: SimpleRNN(32, tanh) → Dense(1).
- **Learning target:** Real value (next number), so we use **MSE** loss (regression).

Imports & Reproducibility

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

np.random.seed(42)
tf.random.set_seed(42)
```

- Sets fixed seeds so training runs are **repeatable** (weights init & batching become deterministic).

Data: Create & Normalize

```
data = np.arange(0, 200, dtype=np.float32) # [0, 1, 2, ..., 199]
max_val = data.max() # 199.0
data_norm = data / max_val # scale to [0, 1]
```

- We generate a **monotonic** sequence of length 200.
- **Normalization** helps training (stabilizes gradients; keeps values in a compact range).
Here: $x_norm = x / 199 \rightarrow$ all inputs/targets lie in $[0, 1]$.

Windowing (Supervised samples from a sequence)

```
window = 5 # timesteps
X, y = [], []
for i in range(len(data_norm) - window):
    X.append(data_norm[i:i+window]) # past 5 values
    y.append(data_norm[i+window]) # next value
X = np.array(X)[..., np.newaxis] # (samples, timesteps=5,
features=1)
y = np.array(y)
```

Why: RNNs expect **sequences of timesteps**. We slide a window of length 5 across the series:

- For positions $i = 0..194$:

- Input $X[i] = [x_i, x_{i+1}, x_{i+2}, x_{i+3}, x_{i+4}]$
- Target $y[i] = x_{i+5}$

• Shapes:

- Number of samples = $200 - 5 = 195$
- X shape = **(195, 5, 1)** $\rightarrow$ (samples, timesteps, features)
(feature dimension is 1 because each timestep is a single scalar)
- y shape = **(195,)**

This turns a 1D series into a supervised learning dataset for **one-step-ahead forecasting**.

Train / Test Split

```
split = int(0.8 * len(X)) # 156
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]
```

- We respect **time order** by taking the **first 80%** as training, **last 20%** as test.
This mimics real forecasting: train on the past, test on later points.
- Result:
 - X_train: (156, 5, 1), y_train: (156,)
 - X_test: (39, 5, 1), y_test: (39,)

Model Architecture

```
model = Sequential([
    SimpleRNN(32, activation="tanh", input_shape=(window, 1)),
    Dense(1)
])
```

- **SimpleRNN(32, tanh):** Processes the 5 steps and outputs the final hidden state (size 32).
 - **tanh** keeps hidden values in **[-1, 1]**, which is common for vanilla RNNs.
- **Dense(1):** Maps the 32-D hidden state to **one real number** (the next value).

Parameter count

- For SimpleRNN(U , $\text{input_dim}=D$), $\text{params} = U * (U + D + 1)$
Here $U=32$, $D=1 \rightarrow 32 * (32+1+1) = 32 * 34 = 1088$
- For Dense(1) after a 32-D input: weights $32 * 1 = 32$ + bias 1 $\rightarrow 33$
- **Total = $1088 + 33 = 1121$ parameters**

Compile (Loss & Optimizer)

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.005)
```

```
, loss="mse" )
```

- **Loss:** mse (mean squared error) — standard for regression.
- **Optimizer:** Adam with a slightly higher LR (0.005) to learn quickly on this simple task.

Train

```
history = model.fit(
    X_train, y_train,
    epochs=300, batch_size=32,
    verbose=0,
    validation_data=(X_test, y_test)
)
```

- **Epochs=300** on a very easy signal is fine; the model will quickly learn the near-linear mapping.
- We track **validation loss** to ensure it generalizes to later timesteps.

Prediction Helper (De-normalize output)

```
def predict_next(seq):
    seq = np.asarray(seq, dtype=np.float32)
    assert len(seq) == window, f"Need {window} numbers"
    seq_norm = (seq / max_val).reshape(1, window, 1) # (1, 5, 1)
    pred_norm = model.predict(seq_norm, verbose=0)[0, 0]
    return pred_norm * max_val # back to
original scale
```

- **Input:** an **unnormalized** list of 5 numbers (e.g., [95,96,97,98,99]).
- We **normalize** to match training scale, predict a **normalized** next value, then **de-normalize**.
- Shapes at inference: (1, 5, 1) → model → (1, 1) → scalar.

Demo Predictions

```
tests = [
    [95, 96, 97, 98, 99], # expect ~100
    [7, 8, 9, 10, 11], # expect ~12
    [100, 101, 102, 103, 104] # expect ~105
```

]

```
for t in tests:
```

```
    print(f"Input: {t} -> Predicted next: {round(predict_next(t))}")
```

- Because the underlying process is **almost linear** ($\text{next} \approx \text{previous} + 1$), a small RNN learns to output $\approx \text{last} + 1$.
- Rounded outputs should be close to 100, 12, 105 respectively.



Transformers

- A **deep learning architecture** introduced in 2017 by Google in the paper “*Attention is All You Need*”.
- Designed for **sequence data** (like text, speech, DNA).
- Became the backbone of **NLP (Natural Language Processing)** and now extends to **vision, speech, and multimodal AI**.

Transformers replaced older sequence models like **RNNs** and **LSTMs** because they are faster, more scalable, and capture long-range dependencies better.

arXiv:1706.03762v5 [cs.CL] 6 Dec 2017

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

1 Introduction

Recurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and

^{*}Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

Transformer Architecture

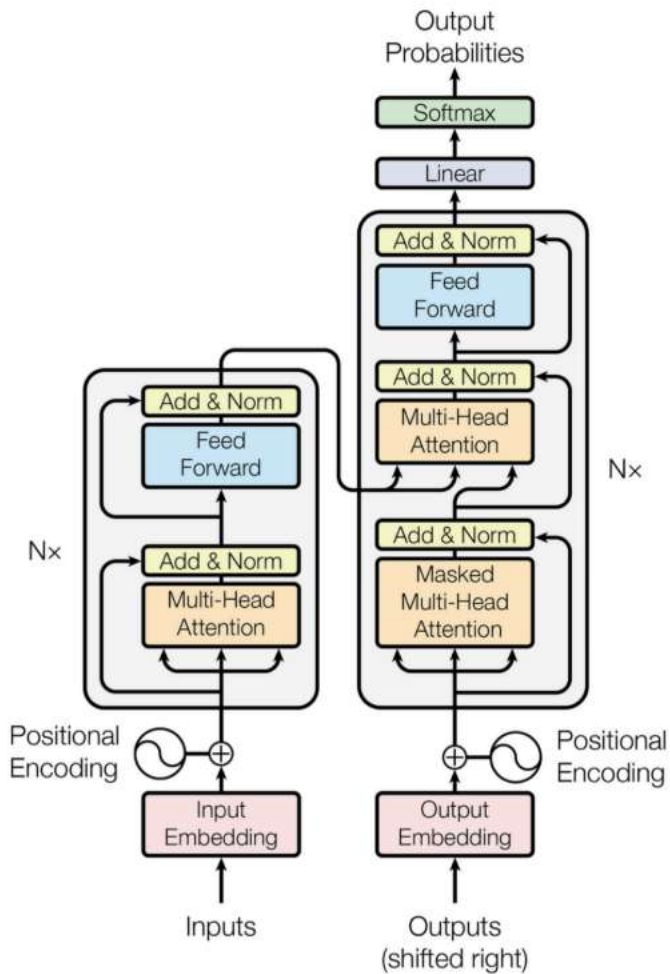


Figure 1: The Transformer - model architecture.

1. Input Embeddings

- Neural networks can't understand text directly. They need **numbers**.
- So each word (or sub-word/token) is converted into a **vector** (embedding).

Example:

Sentence: *"The cat sat"*

- "The" $\rightarrow [0.12, -0.98, 0.45, \dots]$
- "cat" $\rightarrow [0.67, 0.03, -0.45, \dots]$
- "sat" $\rightarrow [-0.22, 0.91, 0.11, \dots]$

These vectors capture meaning — "cat" will be closer to "dog" than to "banana".

2. Positional Encoding

- Unlike RNNs, transformers **don't know word order**.
- They only see a bag of vectors.
- To fix this, we add **positional encodings** (learned vectors).

Example:

Sentence: *"I love Python"*

- Without positions: could be shuffled $\rightarrow$ same meaning for model
- With positions:
 - "I" $\rightarrow$ Position 1
 - "love" $\rightarrow$ Position 2
 - "Python" $\rightarrow$ Position 3

Now the model knows "I love Python" $\neq$ "Python love I".

3. Attention Mechanism

- Attention helps the model decide **which words to focus on** when processing each word.

Example:

Sentence: *"The animal didn't cross the street because it was too wide."*

- What does **"it"** refer to?
- Attention mechanism finds **"street"** is more related to "it" than "animal".

This is how Transformers solve ambiguity.

4. Self-Attention

- Each word looks at **all other words** in the same sentence and updates its meaning.

Example:

Sentence: *"She saw a jaguar."*

- Without self-attention: "jaguar" is unclear (car? animal?).
- With self-attention:
 - If sentence continues: *"She saw a jaguar in the zoo"* → attention links "jaguar" with "zoo" → animal.
 - If sentence continues: *"She saw a jaguar on the highway"* → attention links "jaguar" with "highway" → car.

Context decides meaning.

5. Multi-Head Attention

- Instead of one attention view, we use **multiple heads**.
- Each head learns a different relationship.

Example:

Sentence: *"The cat sat on the mat."*

- Head 1: Focuses on **subject-object** ("cat" ↔ "sat")
- Head 2: Focuses on **preposition** ("sat" ↔ "on")
- Head 3: Focuses on **location** ("on" ↔ "mat")

Like multiple students highlighting different parts of a sentence, then combining notes.

6. Feed-Forward Network

- After attention, each word's vector passes through a **small neural network** (same for every word).
- This refines the representation.

Think of it as polishing the understanding of each word after looking at the context.

7. Layer Normalization + Residual Connections

- To make training stable and efficient:
 - **Residual connections:** Skip connections so information flows easily.
 - **Layer norm:** Normalizes values for faster learning.

Prevents the model from “forgetting” original word meaning after transformations.

8. Stacking Layers

- One layer captures **basic context**.
- Multiple layers stacked = **deep understanding**.

Example:

Sentence: “*The bank approved the loan.*”

- Lower layers: figure out simple word meanings.
- Higher layers: decide **bank = financial institution**, not river bank.

9. Encoder vs Decoder

- **Encoder:** Only reads and understands input. (Used in BERT)
- **Decoder:** Generates words one by one (used in GPT).
- **Encoder–Decoder:** Used for translation (input English → output French).

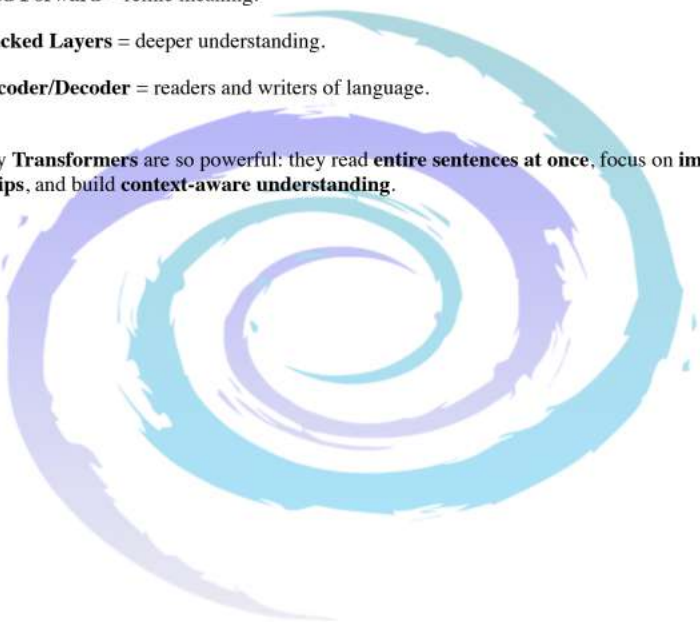
Example (Translation):

- Encoder: “I love dogs” → creates context representation.
- Decoder: “J’aime les chiens” → generates French translation.

Summary

- **Embeddings** = numbers for words.
- **Positional Encoding** = word order.
- **Attention** = what matters most.
- **Self-Attention** = words looking at each other.
- **Multi-Head Attention** = many perspectives at once.
- **Feed Forward** = refine meaning.
- **Stacked Layers** = deeper understanding.
- **Encoder/Decoder** = readers and writers of language.

This is why **Transformers** are so powerful: they read **entire sentences at once**, focus on **important relationships**, and build **context-aware understanding**.



RNN, LSTM vs Transformers

Basic Concept of RNN (Recurrent Neural Network)

- A neural network designed to process **sequential data** (text, speech, time series).
 - Takes input **one step at a time**.
 - Maintains a **hidden state (memory)** that carries info from previous steps.

Example:

Sentence: "I love AI"

- Step 1: Input "I" → produces hidden state h_1
- Step 2: Input "love" + h_1
 h_1 → produces h_2
- Step 3: Input "AI" + h_2
 h_2 → produces h_3

Hidden states pass memory forward, but if the sequence is long, **earlier info is often forgotten**.

Problem with RNNs

- Struggle with **long-term dependencies** (forget distant words).
- Example:
"The book that I borrowed last week was amazing."
 - By the time the model reaches "amazing", it may forget "book".
- Also suffer from **vanishing/exploding gradient problem** during training.

Basic Concept of LSTM (Long Short-Term Memory)

- An advanced RNN architecture (1997).
- Introduced **gates** to control memory flow:
 - **Forget Gate:** Decides what to throw away.
 - **Input Gate:** Decides what new info to store.
 - **Output Gate:** Decides what to send as output.

This helps LSTMs remember **longer sequences** compared to vanilla RNNs.

Example:

Sentence: *"The concert I attended in London last summer was unforgettable."*

- LSTM can connect "concert" ↔ "unforgettable" better than RNN, because it manages memory with gates.

Problems with RNNs (and LSTMs/GRUs)

RNNs process sequences **one word at a time** → like reading slowly word by word.

This caused several issues:

1. Sequential Processing = Slow

- RNNs can't process words in parallel.
- Example: Sentence *"I love Transformers"*
 - RNN reads → "I" → then "love" → then "Transformers".
- For long texts (books, code, big datasets), this is **very slow**.

Transformers process **all words at once** → **parallelism** = faster training.

2. Long-Term Dependencies Problem

- RNNs often **forget context** when sentences get long.
- Example:
"The book that I borrowed from the library last week because it had such good reviews was amazing."
 - RNN may lose track of "book" by the time it reaches "amazing."

Transformers use **self-attention** → any word can directly connect to any other word, no matter how far.

3. Vanishing/Exploding Gradients

- When training RNNs on long sequences, gradients (learning signals) either shrink (vanish) or blow up (explode).
- This makes training unstable.
- LSTMs/GRUs helped, but only partially.

Transformers fix this with **attention + residual connections**, no gradient vanishing issues.

4. Contextual Understanding

- RNN reads left → right (or bidirectional in Bi-LSTM).
- Still limited in capturing **global context**.
- Example: Word “bank”:
 - Sentence: “He sat by the **bank** of the river.”
 - RNN struggles unless trained deeply.
 - Transformer attention → directly links “bank” to “river”.

5. Scalability

- RNNs don't scale well to very large data/models.
- Transformers scale beautifully → that's why we have **GPT-3 (175B params)**, **GPT-4**, **BERT**, etc.

Advantages of Transformers

Parallel processing (train much faster on GPUs).

Global context via self-attention (long-distance relationships).

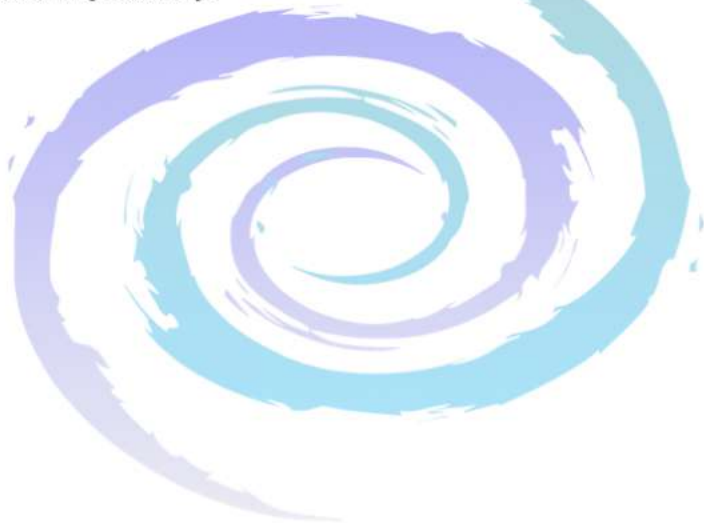
Scalable (works for billions of parameters).

Versatile (works for text, images, audio, even DNA).

State-of-the-art performance → replaced RNNs in NLP.

Simple Analogy

- **RNN/LSTM** = like reading a novel **word by word with one finger**, remembering as best as you can.
- **Transformer** = like looking at the **whole page at once**, highlighting important parts, and instantly understanding relationships.



Encoder vs Decoder for Transformers

Encoder

- **Purpose:** *Understand and represent the input text.*
- **How it works:**
 1. Takes an input sentence (e.g., “The cat sat on the mat”).
 2. Converts each word into embeddings (vectors of numbers).
 3. Uses **self-attention** to understand how each word relates to others.
 - e.g., “bank” → should I connect it with “river” or “money”?
 4. Produces a **contextual representation** of the sentence.
- **Key idea:** It does **not generate new text**, only encodes meaning.
- **Used in:** BERT, translation encoders, text classification.

Think of the encoder as a **reader** who *understands* everything in the input.

Decoder

- **Purpose:** *Generate output step by step.*
- **How it works:**
 1. Starts with encoded information (from encoder, or past tokens if standalone like GPT).
 2. Generates one word at a time → predicts next word.
 3. Uses **self-attention** (to check previous words) + **encoder-decoder attention** (if encoder is present).
 4. Continues until output is complete.
- **Key idea:** It's **generative**.
- **Used in:** GPT (text generation), translation decoders, chatbots.

Think of the decoder as a **writer** who *produces* text word by word.

Putting It Together

- **Encoder-only models:** BERT → good at *understanding*.
- **Decoder-only models:** GPT → good at *generating*.
- **Encoder-Decoder models:** Used in *machine translation* (e.g., Google Translate).
 - Encoder: Understands source language sentence.
 - Decoder: Generates translated sentence in target language.



BERT vs GPT as a transformer based model

BERT (Bidirectional Encoder Representations from Transformers)

- **Created by Google (2018).**
- It is a **transformer-based encoder model**.
- **Key idea:** Reads text **bidirectionally** (from left-to-right and right-to-left at the same time).
- **Goal:** Understand the *meaning of words in context*. For example, the word "bank" could mean a riverbank or a financial bank — BERT figures that out using surrounding words.
- **Uses:**
 - Text classification (spam detection, sentiment analysis)
 - Question answering
 - Named entity recognition
 - Search engines (Google uses BERT to improve search results)

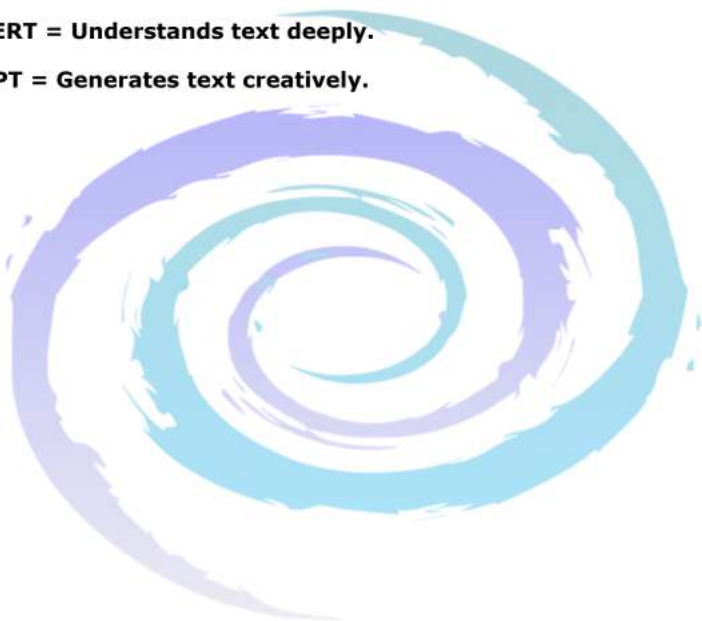
GPT (Generative Pre-trained Transformer)

- **Created by OpenAI (first in 2018, improved versions like GPT-2, GPT-3, GPT-4, GPT-5).**
- It is a **transformer-based decoder model**.
- **Key idea:** Predicts the *next word* in a sentence, one word at a time.
- It is **generative**, meaning it creates new text (stories, code, essays, conversations, etc.).
- **Uses:**
 - Chatbots
 - Text completion
 - Code generation
 - Summarization, translation, content creation

Difference Between BERT and GPT

Feature	BERT	GPT
Architecture	Encoder (Bidirectional)	Decoder (Unidirectional, left → right)
Main Focus	Understanding language (context)	Generating language (text)
Training Task	Masked Language Modeling (predict missing words)	Next Word Prediction (predict next word)
Typical Use Cases	Classification, search, question answering	Chat, text generation, summarization

- **BERT = Understands text deeply.**
- **GPT = Generates text creatively.**



FLAN-T5 = Fine-Tuned Language Net + T5

1. T5 (Text-to-Text Transfer Transformer)

- Introduced by Google in 2019 (*"Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer"*).
- **Encoder-Decoder (Seq2Seq) Transformer** → unlike GPT (decoder-only) or BERT (encoder-only).
- Treats **every NLP task as text-to-text**:
 - Input: "translate English to German: The book is on the table."
 - Output: "Das Buch liegt auf dem Tisch."
- Can handle summarization, translation, classification, QA, etc.

2. FLAN (Fine-tuned Language Net)

- In 2022, Google released **FLAN-T5**: T5 that was **instruction-tuned** on a very large collection of **instructions + tasks**.
- Instead of just pretraining, FLAN-T5 was fine-tuned to **follow natural language instructions** like:
 - "Summarize the following paragraph..."
 - "Translate this sentence into French..."
 - "Classify the sentiment of this review..."

GENAI Project

FLAN-T5 Summarizer & Q&A Assistant

```
#####
#####

#
# Marvellous FLAN-T5 Model
#

#####
#####

import os

os.environ["TOKENIZERS_PARALLELISM"] = "false"

# Import from Hugging Face Transformers
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

# AutoTokenizer : A tokenizer loader that automatically picks the right tokenizer for the model you choose.
# AutoModelForSeq2SeqLM : A pre-trained model loader for Sequence-to-Sequence Language Models (Seq2SeqLM).

# Choose a instruction-tuned model
MODEL_NAME = "google/flan-t5-small"

# A lightweight version of FLAN-T5.
# About 80 million parameters.

print(f"Marvellous FLAN-T5_Summarizer_Q&A_Assistant {MODEL_NAME} model loading...")

# Load tokenizer (handles text <-> tokens).
# AutoTokenizer picks the right tokenizer for the model.
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

# Load the sequence-to-sequence model
model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME)
```

```
#####
#####

#

# This function takes a text prompt (string) uses a FLAN-T5 model to generate a
# continuation/answer and returns the generated text.

#

#####
#####

def Marvellous_run_flan(prompt: str, max_new_tokens: int = 128) -> str:

    # Tokenisation
    # Tokenize the input prompt; return PyTorch tensors; truncate if too long
    inputs = tokenizer(prompt, return_tensors="pt", truncation=True)

    # Generation
    # Generate text from the model with light sampling for naturalness
    outputs = model.generate(
        **inputs,          # pass tokenized inputs (input_ids, attention_mask)
        max_new_tokens=max_new_tokens, # how many new tokens to generate
        do_sample=True,      # enable random sampling
        top_p=0.9,          # nucleus sampling: only consider tokens in the top 90% probability mass
        temperature=0.7     # control randomness (lower = safer/more deterministic)
    )

    # Decode token IDs back into a clean string
    # Example IDs: [71, 867, 1234, 42, 1]
    # Text:      "Hello, how are you?"
    return tokenizer.decode(outputs[0], skip_special_tokens=True).strip()

#####
#####

#

# This function is used for summarisation
```

```
# It creates a prompt with 4 to 6 bullet points
```

```
#
```

```
#####  
#####
```

```
def Marvellous_summarize_text(text: str) -> str:
```

```
    # Prompt template instructing the model to produce 4-6 bullet points
```

```
    prompt = f"Summarize the following text in 4-6 bullet points:\n\n{text}"
```

```
    # Allow a slightly longer output for bullet lists
```

```
    return Marvellous_run_flan(prompt, max_new_tokens=160)
```

```
#####  
#####
```

```
#
```

```
# This function is used to load the contents from our local file
```

```
# And return the complete file contents in one string
```

```
#
```

```
#####  
#####
```

```
def Marvellous_load_context(path: str = "context.txt") -> str:
```

```
    try:
```

```
        # Read the entire file as a single string
```

```
        with open(path, "r", encoding="utf-8") as f:
```

```
            return f.read()
```

```
    except FileNotFoundError:
```

```
        return ""
```

```
#####  
#####
```

```
#
```

```
# This function ask FLAN to answer using ONLY the given context.
```

```
# If answer isn't present, ask it to say 'Not found'.
```

```
#

#####

def Marvellous_answer_from_context(question: str, context: str) -> str:

    if not context.strip():

        return "Context file not found or empty. Create 'context.txt' first."

    # Construct a strict prompt for FLAN-T5

    prompt = (

        "You are a helpful assistant. Answer the question ONLY using the context.\n"

        "If the answer is not in the context, reply exactly: Not found.\n\n"

        f"Context:\n{context}\n\n"

        f"Question: {question}\nAnswer:"

    )

    # Generate a concise answer grounded in the provided notes

    return Marvellous_run_flan(prompt, max_new_tokens=120)

#####

#

# Entry point function

#

#####

def main():

    print("-----")

    print("\n----- Marvellous FLAN-T5 Model -----")

    print("1. Summarize the data")

    print("2. Questions & Answers over local context.txt")

    print("0. Exit")

    print("-----")
```



```
while True:

    choice = input("\nChoose an option (1/2/0): ").strip()

    if choice == "0":

        print("Thank you for using Marvellous FLAN-T5 Model")

        break

    elif choice == "1":

        # Collect multiple lines of text for summarization

        print("You have selected Summarisation option...")

        print("\nPaste text to summarize. End with a blank line:")

        lines = []

        while True:

            line = input()

            # Stop when the user hits Enter on an empty line

            if not line.strip():

                break

            lines.append(line)

        # Join lines into a single block of text

        text = "\n".join(lines).strip()

        # If no text was provided, prompt again

        if not text:

            print("Marvellous FLAN-T5 says : No text received.")

            continue

        # Run summarization and print the result

        print("\nSummary generated by Marvellous FLAN model : ")

        print(Marvellous_summarize_text(text))
```

```

elif choice == "2":

    # Load context from local file 'context.txt'

    ctx = Marvellous_load_context("context.txt")

    if not ctx.strip():

        # Help the user if the context is missing/empty

        print("Missing 'context.txt'. Create it in the same folder and try again.")

        continue

    # Ask a question related to the provided context

    q = input("\nAsk a question about your context to Marvellous FLAN model : ").strip()

    if not q:

        print("No question received.")

        continue

    # Generate an answer grounded only in the context

    print("\nAnswer from Marvellous FLAN model : ")

    print(Marvellous_answer_from_context(q, ctx))

else:

    print("Please choose 1, 2, or 0.")

#####
#####

#

# Starter

#

#####
#####

if __name__ == "__main__":

    main()

```

Large Language Model

- An **artificial intelligence model** trained on **massive amounts of text data** to **understand and generate human-like language**.
- Built using the **Transformer architecture**.
- Called “*Large*” because they have **billions or trillions of parameters** (the “knobs” of a neural network).

Example: GPT-3 has **175 billion** parameters, GPT-4 has **over a trillion (estimated)**.

Internal Working of LLM

1. Training:

- Trained on text from books, websites, code, articles, etc.
- Learns *patterns of language*, not exact memorization.
- Core task: **predict the next word**.
- Example:
Input: “The cat sat on the ...”
Prediction: “mat”.

2. Embeddings:

- Converts words/tokens into **vectors (numbers)**.
- Example: “dog” is close to “cat” in vector space, far from “banana”.

3. Self-Attention (Transformer core):

- Model can relate any word to any other word in the sequence.
- Example: In “*The book that I read yesterday was amazing*”, the model links **book** ↔ **amazing**.

4. Fine-Tuning:

- After pretraining, models are often fine-tuned:
 - **Instruction tuning** → make it follow user commands.
 - **RLHF (Reinforcement Learning with Human Feedback)** → align it with human expectations.

Actual Meaning of Large

- The **size of the model (parameters)** allows it to store rich language patterns.
- Small models: Learn basic grammar, short context.
- Large models: Learn reasoning, long-term dependencies, creativity.

More parameters + more data = better understanding and generation.

Capabilities of LLMs

Language Understanding → summarization, Q&A, sentiment analysis.

Text Generation → essays, stories, reports, scripts.

Translation → English ⇌ French ⇌ Hindi, etc.

Coding → generate/debug code (e.g., Copilot, ChatGPT Code Interpreter).

Reasoning → solve logical/mathematical problems.

Multimodal extensions → some LLMs handle **text + images + audio** (e.g., GPT-4, Gemini).

Examples of LLMs

- **OpenAI** → GPT-3, GPT-4, GPT-5 (used in ChatGPT).
- **Google** → PaLM, Gemini.
- **Meta** → LLaMA series.
- **Anthropic** → Claude series.
- **Mistral** → Mistral 7B, Mixtral.

Analogy

Imagine teaching a **child to speak**:

- At first, they read a **lot of books** (pretraining).
- Then they practice **Q&A with a teacher** (fine-tuning).
- Finally, they learn **manners, safety, and alignment** with society (RLHF).

That's exactly how LLMs evolve.

Summary

- **LLMs are AI models trained on huge text data to understand and generate human-like text.**
- They are built on **Transformers**, use **billions/trillions of parameters**, and power applications like **ChatGPT, Claude, Gemini, Copilot**.
- They excel at **understanding, generating, translating, reasoning**, but also have **limitations** like hallucination and bias.

RLHF Workflow



Handwritten Digits Classification using Deep Learning Techniques

```

import os
import argparse
import random
from typing import Tuple

import numpy as np
import matplotlib.pyplot as plt

# TensorFlow / Keras
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Sklearn for metrics and baselines
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier

#
=====
# Global config
#
=====
SEED = 42
DIGIT_CLASSES = [str(i) for i in range(10)]
ARTIFACT_DIR = "artifacts_digits"
BEST_MODEL = os.path.join(ARTIFACT_DIR, "digits_cnn.h5")
FINAL_MODEL = os.path.join(ARTIFACT_DIR, "digits_cnn_final.h5")

#
=====
# Utils
#
=====
#####

# Function: set_seed
# Inputs: seed (int) - random seed value
# Outputs: None
# Description: Sets seed for Python, NumPy, and TensorFlow to ensure reproducibility.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def set_seed(seed: int = SEED):
    os.environ["TF_DETERMINISTIC_OPS"] = "1"

```



```

random.seed(seed)
np.random.seed(seed)
tf.random.set_seed(seed)

#####
# Function: ensure_dir
# Inputs: path (str) - directory path
# Outputs: None
# Description: Creates directory if it does not exist.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####
def ensure_dir(path: str):
    os.makedirs(path, exist_ok=True)

#####

# Function: plot_training_curves
# Inputs: history (keras.callbacks.History), out_dir (str)
# Outputs: Saves accuracy and loss curve plots
# Description: Plots and saves training/validation accuracy and loss curves.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####
def plot_training_curves(history, out_dir=ARTIFACT_DIR):
    ensure_dir(out_dir)
    # Accuracy
    plt.figure(figsize=(7, 5))
    plt.plot(history.history["accuracy"], label="train_acc")
    plt.plot(history.history["val_accuracy"], label="val_acc")
    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")
    plt.title("Training vs Validation Accuracy")
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(out_dir, "acc_curve.png"))
    plt.close()

    # Loss
    plt.figure(figsize=(7, 5))
    plt.plot(history.history["loss"], label="train_loss")
    plt.plot(history.history["val_loss"], label="val_loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title("Training vs Validation Loss")
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(out_dir, "loss_curve.png"))
    plt.close()

#####
# Function: _annotate_confmat
# Inputs: ax (matplotlib.axes.Axes), cm (np.ndarray), fmt (str)
# Outputs: Annotated confusion matrix
# Description: Adds numeric annotations to confusion matrix heatmap.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025

```

```
#####

def _annotate_confmat(ax, cm, fmt="{:.2f}"):
    n, m = cm.shape
    thresh = cm.max() / 2.0
    for i in range(n):
        for j in range(m):
            val = cm[i, j]
            color = "white" if val > thresh else "black"
            txt = f"{val:.2f}" if isinstance(val, float) else f"{val:d}"
            ax.text(j, i, txt, ha="center", va="center", color=color, fontsize=8)

#####

# Function: plot_confusion_matrix
# Inputs: y_true (np.ndarray), y_pred (np.ndarray), out_dir (str), normalize (bool)
# Outputs: Saves confusion matrix plot
# Description: Generates and saves confusion matrix as image.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def plot_confusion_matrix(y_true, y_pred, out_dir=ARTIFACT_DIR, normalize=True):
    ensure_dir(out_dir)
    cm = confusion_matrix(y_true, y_pred)
    if normalize:
        cm = cm.astype("float") / cm.sum(axis=1, keepdims=True)

    fig, ax = plt.subplots(figsize=(8.5, 7))
    im = ax.imshow(cm, interpolation="nearest", cmap="Blues")
    ax.figure.colorbar(im, ax=ax)
    ax.set(
        xticks=np.arange(len(DIGIT_CLASSES)),
        yticks=np.arange(len(DIGIT_CLASSES)),
        xticklabels=DIGIT_CLASSES,
        yticklabels=DIGIT_CLASSES,
        ylabel="True label",
        xlabel="Predicted label",
        title="Confusion Matrix (normalized)" if normalize else "Confusion Matrix",
    )
    plt.setp(ax.get_xticklabels(), rotation=45, ha="right", rotation_mode="anchor")
    _annotate_confmat(ax, cm)
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, "confusion_matrix.png"))
    plt.close(fig)

#####

# Function: save_classification_report
# Inputs: y_true (np.ndarray), y_pred (np.ndarray), out_dir (str)
# Outputs: Saves classification report
# Description: Prints and saves classification report with precision, recall, and F1 score.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def save_classification_report(y_true, y_pred, out_dir=ARTIFACT_DIR):
    ensure_dir(out_dir)
```

```

report = classification_report(
    y_true, y_pred, target_names=DIGIT_CLASSES, digits=4
)
print(report)
with open(os.path.join(out_dir, "classification_report.txt"), "w") as f:
    f.write(report)

#####

# Function: show_misclassifications
# Inputs: x (np.ndarray), y_true (np.ndarray), y_pred (np.ndarray), limit (int), out_dir (str)
# Outputs: Saves misclassified images grid
# Description: Displays and saves examples of misclassified test images.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def show_misclassifications(x, y_true, y_pred, limit=25, out_dir=ARTIFACT_DIR):
    ensure_dir(out_dir)
    wrong = np.where(y_true != y_pred)[0]
    if len(wrong) == 0:
        print("No misclassifications")
        return
    sel = wrong[:limit]
    cols = 5
    rows = int(np.ceil(len(sel) / cols))
    plt.figure(figsize=(12, 2.6 * rows))
    for i, idx in enumerate(sel, 1):
        img = x[idx].squeeze()
        plt.subplot(rows, cols, i)
        plt.imshow(img, cmap="gray")
        plt.title(f'T:{DIGIT_CLASSES[y_true[idx]]}\n{P:{DIGIT_CLASSES[y_pred[idx]]}', fontsize=9)
        plt.axis("off")
    plt.tight_layout()
    plt.savefig(os.path.join(out_dir, "misclassifications.png"))
    plt.close()

#####

# Function: save_label_map
# Inputs: out_dir (str)
# Outputs: Saves label mapping file
# Description: Saves numeric-to-class name mapping (0-9) for MNIST digits.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def save_label_map(out_dir=ARTIFACT_DIR):
    ensure_dir(out_dir)
    with open(os.path.join(out_dir, "label_map.txt"), "w") as f:
        for i, name in enumerate(DIGIT_CLASSES):
            f.write(f'{i}: {name}\n')

#####

# Function: save_summary
# Inputs: test_acc (float), test_loss (float), epochs (int), out_dir (str)

```



```
# Outputs: Saves summary text file
# Description: Saves final model performance summary and artifact list.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def save_summary(test_acc, test_loss, epochs, out_dir=ARTIFACT_DIR):
    ensure_dir(out_dir)
    with open(os.path.join(out_dir, "summary.txt"), "w") as f:
        f.write(
            "MNIST Digits CNN Summary\n"
            "=====\n"
            f"Test Accuracy: {test_acc:.4f}\n"
            f"Test Loss : {test_loss:.4f}\n"
            f"Epochs : {epochs}\n"
            "Artifacts : acc_curve.png, loss_curve.png, confusion_matrix.png,\n"
            "              misclassifications.png, classification_report.txt,\n"
            "              label_map.txt, digits_cnn.h5, digits_cnn_final.h5\n"
        )

# =====
# Data
# =====
#####

# Function: load_data
# Inputs: val_split (float) - validation split ratio
# Outputs: (train_data, val_data, test_data) tuples
# Description: Loads MNIST dataset, normalizes and splits into train, val, test sets.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def load_data(val_split=0.1) -> Tuple[Tuple[np.ndarray, np.ndarray],
                                     Tuple[np.ndarray, np.ndarray],
                                     Tuple[np.ndarray, np.ndarray]]:
    (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
    x_train, x_val, y_train, y_val = train_test_split(
        x_train, y_train, test_size=val_split, random_state=SEED, stratify=y_train
    )
    # Scale and add channel dim
    x_train = (x_train.astype("float32") / 255.0)[..., None]
    x_val = (x_val.astype("float32") / 255.0)[..., None]
    x_test = (x_test.astype("float32") / 255.0)[..., None]
    return (x_train, y_train), (x_val, y_val), (x_test, y_test)

#####

# Function: load_flattened
# Inputs: None
# Outputs: Flattened train and test arrays with labels
# Description: Loads MNIST and flattens images for classical ML models.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def load_flattened():
    (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
```

```
x_train = x_train.reshape(len(x_train), -1).astype("float32") / 255.0
x_test = x_test.reshape(len(x_test), -1).astype("float32") / 255.0
return x_train, y_train, x_test, y_test
```

```
# =====
# Model
# =====
#####

# Function: build_cnn
# Inputs: lr (float) - learning rate
# Outputs: Compiled Keras CNN model
# Description: Builds and compiles CNN with augmentation, Conv2D, Dropout, BatchNorm, Dense.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def build_cnn(lr=1e-3) -> keras.Model:
    data_augmentation = keras.Sequential(
        [
            layers.RandomTranslation(0.05, 0.05, fill_mode="nearest"),
            layers.RandomRotation(0.05, fill_mode="nearest"),
            layers.RandomZoom(0.05, 0.05, fill_mode="nearest"),
        ],
        name="augmentation",
    )

    inputs = keras.Input(shape=(28, 28, 1))
    x = data_augmentation(inputs)

    x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)
    x = layers.BatchNormalization()(x)
    x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)
    x = layers.Dropout(0.25)(x)

    x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
    x = layers.BatchNormalization()(x)
    x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)
    x = layers.Dropout(0.25)(x)

    x = layers.Flatten()(x)
    x = layers.Dense(128, activation="relu")(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.40)(x)

    outputs = layers.Dense(10, activation="softmax")(x)

    model = keras.Model(inputs, outputs, name="digits_cnn")
    opt = keras.optimizers.Adam(learning_rate=lr)
    model.compile(optimizer=opt, loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return model
```

```
# =====
# Train / Evaluate
# =====
#####
```

```

# Function: train_and_evaluate
# Inputs: batch_size (int), epochs (int), lr (float)
# Outputs: Trained model + saved artifacts
# Description: Trains CNN on MNIST, evaluates, saves curves, confusion matrix, misclassifications,
report.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def train_and_evaluate(batch_size=128, epochs=10, lr=1e-3):
    ensure_dir(ARTIFACT_DIR)
    (x_train, y_train), (x_val, y_val), (x_test, y_test) = load_data(val_split=0.1)
    model = build_cnn(lr=lr)
    model.summary()

    callbacks = [
        keras.callbacks.ModelCheckpoint(BEST_MODEL, monitor="val_accuracy",
save_best_only=True, verbose=1),
        keras.callbacks.EarlyStopping(patience=3, restore_best_weights=True, monitor="val_loss"),
        keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=2, min_lr=1e-5, verbose=1),
    ]

    history = model.fit(
        x_train, y_train,
        validation_data=(x_val, y_val),
        batch_size=batch_size,
        epochs=epochs,
        callbacks=callbacks,
        verbose=2
    )

    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
    print(f"[TEST] loss: {test_loss:.4f} | acc: {test_acc:.4f}")

    model.save(FINAL_MODEL)

    # Metrics & artifacts
    plot_training_curves(history, ARTIFACT_DIR)
    y_pred = model.predict(x_test, batch_size=256).argmax(axis=1)
    plot_confusion_matrix(y_test, y_pred, ARTIFACT_DIR, normalize=True)
    save_classification_report(y_test, y_pred, ARTIFACT_DIR)
    show_misclassifications(x_test, y_test, y_pred, limit=25, out_dir=ARTIFACT_DIR)
    save_label_map(ARTIFACT_DIR)
    save_summary(test_acc, test_loss, len(history.history["loss"]), ARTIFACT_DIR)

# =====
# Inference
# =====
#####

# Function: inference_grid
# Inputs: n_samples (int), seed (int)
# Outputs: Saved inference grid image
# Description: Loads saved model, predicts on random test samples, saves predictions grid.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

```



```
def inference_grid(n_samples=9, seed=7):
    if not os.path.exists(BEST_MODEL):
        print(f"Could not find {BEST_MODEL}. Train first with --train.")
        return
    (_, _), (x_test, y_test) = keras.datasets.mnist.load_data()
    x_test = (x_test.astype("float32") / 255.0)[..., None]

    model = keras.models.load_model(BEST_MODEL)

    rng = np.random.default_rng(seed)
    idx = rng.choice(len(x_test), size=n_samples, replace=False)

    imgs = x_test[idx]
    labs = y_test[idx]
    preds = model.predict(imgs, verbose=0).argmax(axis=1)

    cols = int(np.ceil(np.sqrt(n_samples)))
    rows = int(np.ceil(n_samples / cols))
    plt.figure(figsize=(2.8 * cols, 2.8 * rows))
    for i in range(n_samples):
        plt.subplot(rows, cols, i + 1)
        plt.imshow(imgs[i].squeeze(), cmap="gray")
        plt.title(f"P:{DIGIT_CLASSES[preds[i]]}\nT:{DIGIT_CLASSES[labs[i]]}", fontsize=9)
        plt.axis("off")
    plt.tight_layout()
    ensure_dir(ARTIFACT_DIR)
    out_path = os.path.join(ARTIFACT_DIR, "inference_grid.png")
    plt.savefig(out_path)
    plt.close()
    print("Saved:", out_path)

# =====
# Baselines
# =====
#####

# Function: run_baselines
# Inputs: None
# Outputs: Prints accuracy of LogReg, LinearSVC, RandomForest
# Description: Runs classical ML baselines on flattened MNIST digits.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def run_baselines():
    x_train, y_train, x_test, y_test = load_flattened()

    # Logistic Regression
    logreg = Pipeline([
        ("scaler", StandardScaler(with_mean=True)),
        ("clf", LogisticRegression(max_iter=200, n_jobs=-1))
    ])
    logreg.fit(x_train, y_train)
    print("LogReg acc:", accuracy_score(y_test, logreg.predict(x_test)))

    # Linear SVM
    svm = Pipeline([
        ("scaler", StandardScaler(with_mean=True)),
        ("clf", LinearSVC(C=1.0, dual=True, max_iter=5000))
    ])
```

```

    ])
    svm.fit(x_train, y_train)
    print("LinearSVC acc:", accuracy_score(y_test, svm.predict(x_test)))

# Random Forest
    rf = RandomForestClassifier(n_estimators=200, max_depth=None, n_jobs=-1,
random_state=SEED)
    rf.fit(x_train, y_train)
    print("RandomForest acc:", accuracy_score(y_test, rf.predict(x_test)))

# =====
# CLI
# =====
#####

# Function: parse_args
# Inputs: None
# Outputs: Parsed CLI arguments
# Description: Defines command-line arguments for training, inference, baselines.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def parse_args():
    p = argparse.ArgumentParser(description="MNIST Digits Classification Case Study")
    p.add_argument("--train", action="store_true", help="Train the CNN and save artifacts.")
    p.add_argument("--infer", action="store_true", help="Generate an inference grid using saved
model.")
    p.add_argument("--samples", type=int, default=9, help="Number of samples for inference
grid.")
    p.add_argument("--baselines", action="store_true", help="Run classical ML baselines.")
    p.add_argument("--epochs", type=int, default=10, help="Epochs for CNN training.")
    p.add_argument("--batch", type=int, default=128, help="Batch size for CNN training.")
    p.add_argument("--lr", type=float, default=1e-3, help="Learning rate for Adam.")
    return p.parse_args()

# =====
# Entry point
# =====
#####

# Function: main
# Inputs: None
# Outputs: None
# Description: Entry point. Parses CLI args, runs train/infer/baselines accordingly.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def main():
    set_seed(SEED)
    ensure_dir(ARTIFACT_DIR)
    args = parse_args()

    did_anything = False
    if args.train:
        train_and_evaluate(batch_size=args.batch, epochs=args.epochs, lr=args.lr)
        did_anything = True

```

```

if args.infer:
    inference_grid(n_samples=args.samples)
    did_anything = True
if args.baselines:
    run_baselines()
    did_anything = True

if not did_anything:
    print(
        "Nothing to do. Try one of:\n"
        " python digits_classification_case_study.py --train\n"
        " python digits_classification_case_study.py --infer --samples 9\n"
        " python digits_classification_case_study.py --baselines\n"
    )

```

```

#####
if __name__ == "__main__":
    main()
#####

```



Apparel Classification System using Deep Learning

```
#####
# Function: load_data
# Inputs: val_split (float) - validation split ratio
# Outputs: (train_data, val_data, test_data) tuples
# Description: Loads Fashion-MNIST dataset, normalizes and splits into train, val, test sets.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def load_data(val_split=0.1) -> Tuple[Tuple[np.ndarray, np.ndarray],
                                     Tuple[np.ndarray, np.ndarray],
                                     Tuple[np.ndarray, np.ndarray]]:
    (x_train, y_train), (x_test, y_test) = keras.datasets.fashion_mnist.load_data()
    x_train, x_val, y_train, y_val = train_test_split(
        x_train, y_train, test_size=val_split, random_state=SEED, stratify=y_train
    )
    x_train = (x_train.astype("float32") / 255.0)[..., None]
    x_val = (x_val.astype("float32") / 255.0)[..., None]
    x_test = (x_test.astype("float32") / 255.0)[..., None]
    return (x_train, y_train), (x_val, y_val), (x_test, y_test)

#####
# Function: load_flattened
# Inputs: None
# Outputs: Flattened train and test arrays with labels
# Description: Loads Fashion-MNIST and flattens images for classical ML models.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def load_flattened():
    (x_train, y_train), (x_test, y_test) = keras.datasets.fashion_mnist.load_data()
    x_train = x_train.reshape(len(x_train), -1).astype("float32") / 255.0
    x_test = x_test.reshape(len(x_test), -1).astype("float32") / 255.0
    return x_train, y_train, x_test, y_test

#####
# Function: build_cnn
# Inputs: lr (float) - learning rate
# Outputs: Compiled Keras CNN model
# Description: Builds and compiles CNN with augmentation, Conv2D, Dropout, BatchNorm, Dense.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def build_cnn(lr=1e-3) -> keras.Model:
    data_augmentation = keras.Sequential(
        [
            layers.RandomTranslation(0.05, 0.05, fill_mode="nearest"),
            layers.RandomRotation(0.05, fill_mode="nearest"),
            layers.RandomZoom(0.05, 0.05, fill_mode="nearest"),
        ],
    )
```



```

        name="augmentation",
    )
    inputs = keras.Input(shape=(28, 28, 1))
    x = data_augmentation(inputs)
    x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)
    x = layers.BatchNormalization()(x)
    x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)
    x = layers.Dropout(0.25)(x)

    x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
    x = layers.BatchNormalization()(x)
    x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)
    x = layers.Dropout(0.25)(x)

    x = layers.Flatten()(x)
    x = layers.Dense(128, activation="relu")(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.40)(x)
    outputs = layers.Dense(10, activation="softmax")(x)

    model = keras.Model(inputs, outputs, name="fashion_cnn")
    opt = keras.optimizers.Adam(learning_rate=lr)
    model.compile(optimizer=opt, loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return model

#####
# Function: train_and_evaluate
# Inputs: batch_size (int), epochs (int), lr (float)
# Outputs: Trained model + saved artifacts
# Description: Trains CNN on Fashion-MNIST, evaluates, saves curves, confusion matrix, misclassifications,
report.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####

def train_and_evaluate(batch_size=128, epochs=15, lr=1e-3):
    ensure_dir(ARTIFACT_DIR)
    (x_train, y_train), (x_val, y_val), (x_test, y_test) = load_data(val_split=0.1)
    model = build_cnn(lr=lr)
    model.summary()
    callbacks = [
        keras.callbacks.ModelCheckpoint(BEST_MODEL, monitor="val_accuracy", save_best_only=True,
        verbose=1),
        keras.callbacks.EarlyStopping(patience=4, restore_best_weights=True, monitor="val_loss"),
        keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=2, min_lr=1e-5, verbose=1),
    ]
    history = model.fit(
        x_train, y_train,
        validation_data=(x_val, y_val),
        batch_size=batch_size,
        epochs=epochs,
        callbacks=callbacks,
        verbose=2
    )

```

```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"[TEST] loss: {test_loss:.4f} | acc: {test_acc:.4f}")

model.save(FINAL_MODEL)
plot_training_curves(history, ARTIFACT_DIR)
y_pred = model.predict(x_test, batch_size=256).argmax(axis=1)
plot_confusion_matrix(y_test, y_pred, ARTIFACT_DIR, normalize=True)
save_classification_report(y_test, y_pred, ARTIFACT_DIR)
show_misclassifications(x_test, y_test, y_pred, limit=25, out_dir=ARTIFACT_DIR)
save_label_map(ARTIFACT_DIR)
save_summary(test_acc, test_loss, len(history.history["loss"]), ARTIFACT_DIR)
```

```
#####
# Function: inference_grid
# Inputs: n_samples (int), seed (int)
# Outputs: Saved inference grid image
# Description: Loads saved model, predicts on random test samples, saves predictions grid.
# Author: Piyush Manohar Khairnar
# Date: 15/08/2025
#####
```

```
def inference_grid(n_samples=9, seed=7):
    if not os.path.exists(BEST_MODEL):
        print(f"Could not find {BEST_MODEL}. Train first with --train.")
        return
    (_, _), (x_test, y_test) = keras.datasets.fashion_mnist.load_data()
    x_test = x_test.astype("float32") / 255.0[... , None]
    model = keras.models.load_model(BEST_MODEL)

    rng = np.random.default_rng(seed)
    idx = rng.choice(len(x_test), size=n_samples, replace=False)
    imgs = x_test[idx]
    labs = y_test[idx]
    preds = model.predict(imgs, verbose=0).argmax(axis=1)

    cols = int(np.ceil(np.sqrt(n_samples)))
    rows = int(np.ceil(n_samples / cols))
    plt.figure(figsize=(2.8 * cols, 2.8 * rows))
    for i in range(n_samples):
        plt.subplot(rows, cols, i + 1)
        plt.imshow(imgs[i].squeeze(), cmap="gray")
        plt.title(f"P: {FASHION_CLASSES[preds[i]]}\nT: {FASHION_CLASSES[labs[i]]}", fontsize=9)
        plt.axis("off")
    plt.tight_layout()
    ensure_dir(ARTIFACT_DIR)
    out_path = os.path.join(ARTIFACT_DIR, "inference_grid.png")
    plt.savefig(out_path)
    plt.close()
    print("Saved:", out_path)
```

```
#####
# Function: run_baselines
# Inputs: None
# Outputs: Prints accuracy of LogReg, LinearSVC, RandomForest
```


Description: Runs classical ML baselines on flattened Fashion-MNIST.

Author: Piyush Manohar Khairnar

Date: 15/08/2025

#####

def run_baselines():

 x_train, y_train, x_test, y_test = load_flattened()

 logreg = Pipeline([

 ("scaler", StandardScaler(with_mean=True)),

 ("clf", LogisticRegression(max_iter=200, n_jobs=-1))

])

 logreg.fit(x_train, y_train)

 print("LogReg acc:", accuracy_score(y_test, logreg.predict(x_test)))

 svm = Pipeline([

 ("scaler", StandardScaler(with_mean=True)),

 ("clf", LinearSVC(C=1.0, dual=True, max_iter=5000))

])

 svm.fit(x_train, y_train)

 print("LinearSVC acc:", accuracy_score(y_test, svm.predict(x_test)))

 rf = RandomForestClassifier(n_estimators=200, max_depth=None, n_jobs=-1, random_state=SEED)

 rf.fit(x_train, y_train)

 print("RandomForest acc:", accuracy_score(y_test, rf.predict(x_test)))

#####

Function: parse_args

Inputs: None

Outputs: Parsed CLI arguments

Description: Defines command-line arguments for training, inference, baselines.

Author: Piyush Manohar Khairnar

Date: 15/08/2025

#####

def parse_args():

 p = argparse.ArgumentParser(description="Fashion-MNIST Case Study")

 p.add_argument("--train", action="store_true", help="Train the CNN and save artifacts.")

 p.add_argument("--infer", action="store_true", help="Generate an inference grid using saved model.")

 p.add_argument("--samples", type=int, default=9, help="Number of samples for inference grid.")

 p.add_argument("--baselines", action="store_true", help="Run classical ML baselines.")

 p.add_argument("--epochs", type=int, default=15, help="Epochs for CNN training.")

 p.add_argument("--batch", type=int, default=128, help="Batch size for CNN training.")

 p.add_argument("--lr", type=float, default=1e-3, help="Learning rate for Adam.")

 return p.parse_args()

#####

Function: main

Inputs: None

Outputs: None

Description: Entry point. Parses CLI args, runs train/infer/baselines accordingly.

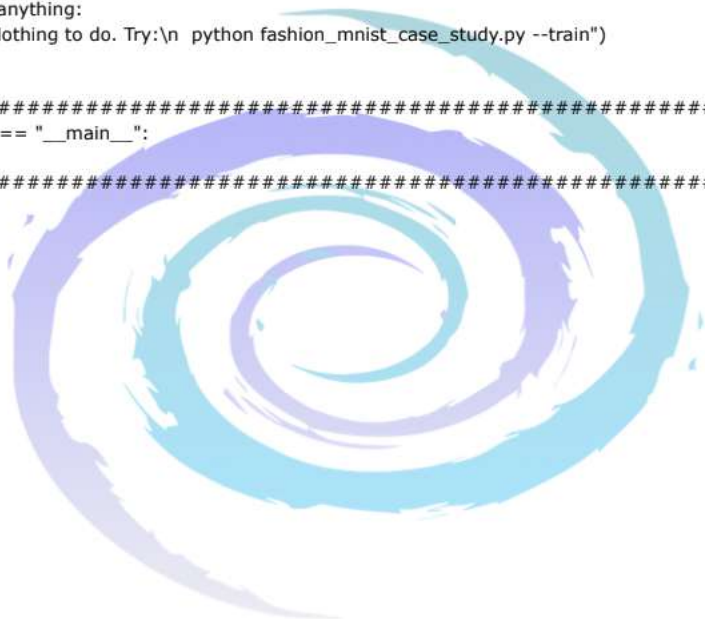
Author: Piyush Manohar Khairnar

Date: 15/08/2025

#####

```
def main():
    set_seed(SEED)
    ensure_dir(ARTIFACT_DIR)
    args = parse_args()
    did_anything = False
    if args.train:
        train_and_evaluate(batch_size=args.batch, epochs=args.epochs, lr=args.lr)
        did_anything = True
    if args.infer:
        inference_grid(n_samples=args.samples)
        did_anything = True
    if args.baselines:
        run_baselines()
        did_anything = True
    if not did_anything:
        print("Nothing to do. Try:\n python fashion_mnist_case_study.py --train")
```

```
#####
if __name__ == "__main__":
    main()
#####
```



The above project contains :

1. Utility Functions

- `set_seed`, `ensure_dir`, `plot_training_curves`, `plot_confusion_matrix`, `save_classification_report`, `show_misclassifications`, `save_label_map`, `save_summary`

2. Data Loaders

- `load_data` → loads + normalizes dataset with train/val/test split
- `load_flattened` → prepares flattened arrays for classical ML baselines

3. Model

- `build_cnn` → defines and compiles a CNN with augmentation

4. Training & Evaluation

- `train_and_evaluate` → trains CNN, evaluates on test set, saves model + artifacts

5. Inference

- `inference_grid` → loads saved model and shows predictions for random samples

6. Baselines

- `run_baselines` → Logistic Regression, Linear SVM, Random Forest

7. CLI

- `parse_args` → command line flags (`--train`, `--infer`, `--baselines`)
- `main` → entry point to run training, inference, or baselines

Please follow below steps :

1. Save the code

Marvellous_fashion_mnist_case_study.py

2. Install dependencies

Open a terminal and create a virtual environment (recommended):

```
python -m venv venv
```

Activate it:

- **Windows:**
venv\Scripts\activate
- **Mac/Linux:**
source venv/bin/activate

Now install the required libraries:

```
pip install tensorflow scikit-learn matplotlib numpy pandas tqdm
```

3. Train the CNN

```
python Marvellous_fashion_mnist_case_study.py --train
```

This will:

- Train the CNN model
- Save the best model in artifacts/fashion_cnn.h5
- Save training curves (acc_curve.png, loss_curve.png)
- Save confusion matrix, misclassifications, classification report, etc.

All outputs will be in the artifacts/ folder.

4. Run Inference (predict on random test images)

After training, you can generate an inference grid:

```
python Marvellousfashion_mnist_case_study.py --infer --samples 9
```

This will create artifacts/inference_grid.png showing predictions vs true labels.

5. Try classical baselines (Logistic Regression, SVM, Random Forest)

```
python Marvellousfashion_mnist_case_study.py --baselines
```

This compares simple ML methods vs CNN.

Outputs

All results go into the artifacts/ folder:

- fashion_cnn.h5 → best model
- fashion_cnn_final.h5 → final model
- acc_curve.png, loss_curve.png
- confusion_matrix.png
- misclassifications.png
- classification_report.txt
- summary.txt
- label_map.txt
- inference_grid.png

